COPYRIGHT WARNING: Copyright in these original lectures is owned by Monash University. You may transcribe, take notes, download or stream lectures for the purpose of your research and study only. If used for any other purpose, (excluding exceptions in the Copyright Act 1969 (Cth)) the University may take legal action for infringement of copyright.

Do not share, redistribute, or upload the lecture to a third party without a written permission!

FIT3181/5215 Deep Learning

Week 11: Deep Generative Models

CE/Lecturer: Dr Trung Le (Clayton)

Lecturer: Dr Binh Nguyen (Clayton)

Lecturer: Dr Ting Fung Fung (Malaysia)

Lecturer: Dr Arghya Pal (Malaysia)

Lecturer: Dr Sailaja Rajanala (Malaysia)

Email: trunglm@monash.edu

GROUP OF EIGHT AUSTRALIA

Department of Data Science and AI
Faculty of Information Technology, Monash University, Australia

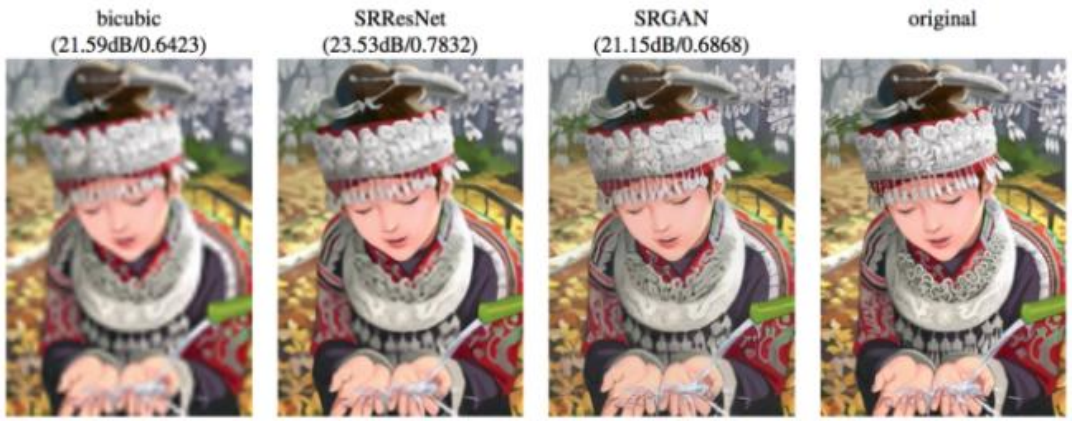


Generative AI teaser: pre-2022



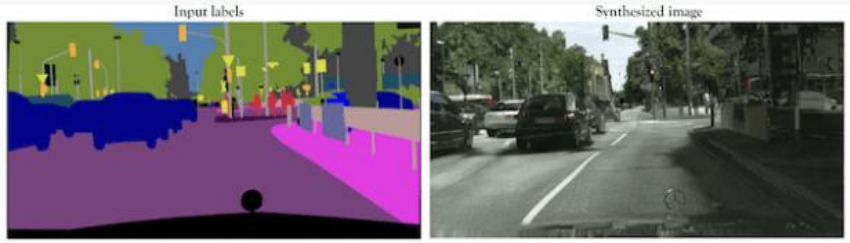
Realistic Image Generation

Low *High*



Super Resolution

Sketch, sematic image *Realistic image*



Semantic-Image-to-Photo Translation



Text to Image

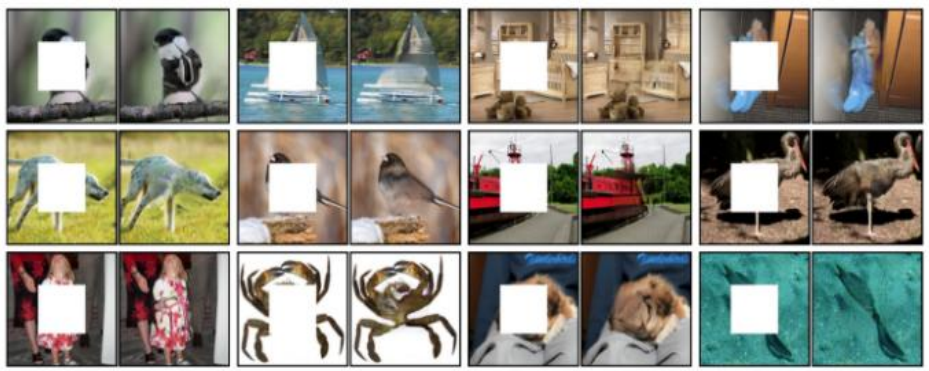


Image Painting

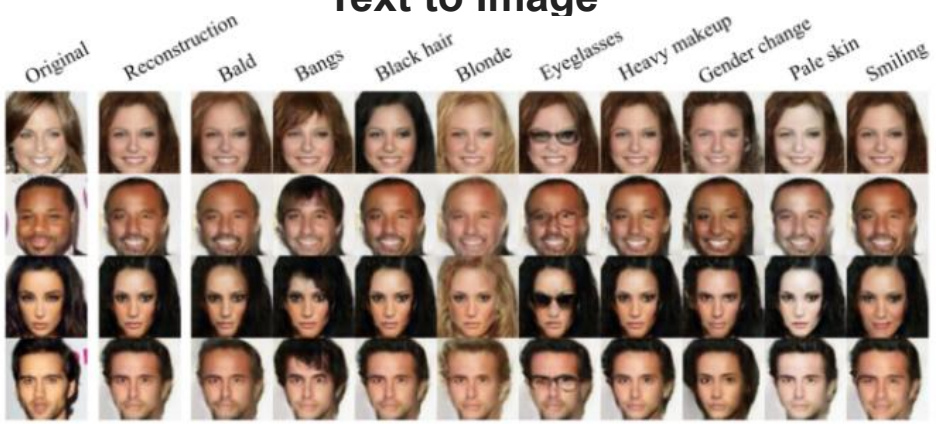
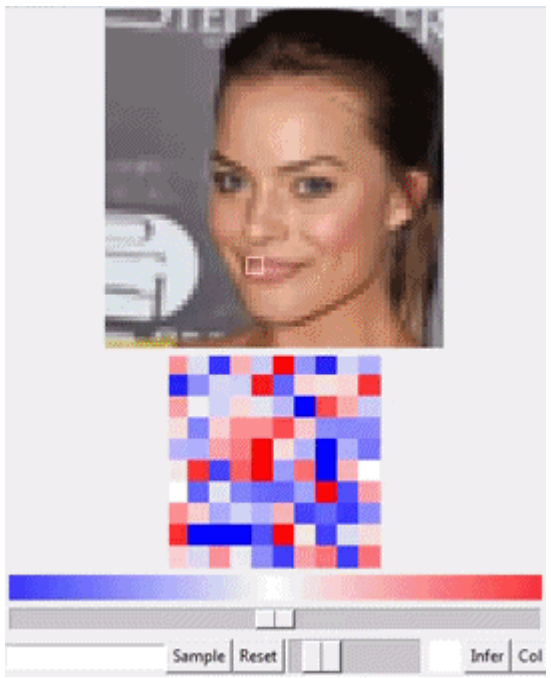


Image Editing

Generative AI teaser: pre-2022, GAN-based



Style Transfer



Neural Photo Editor

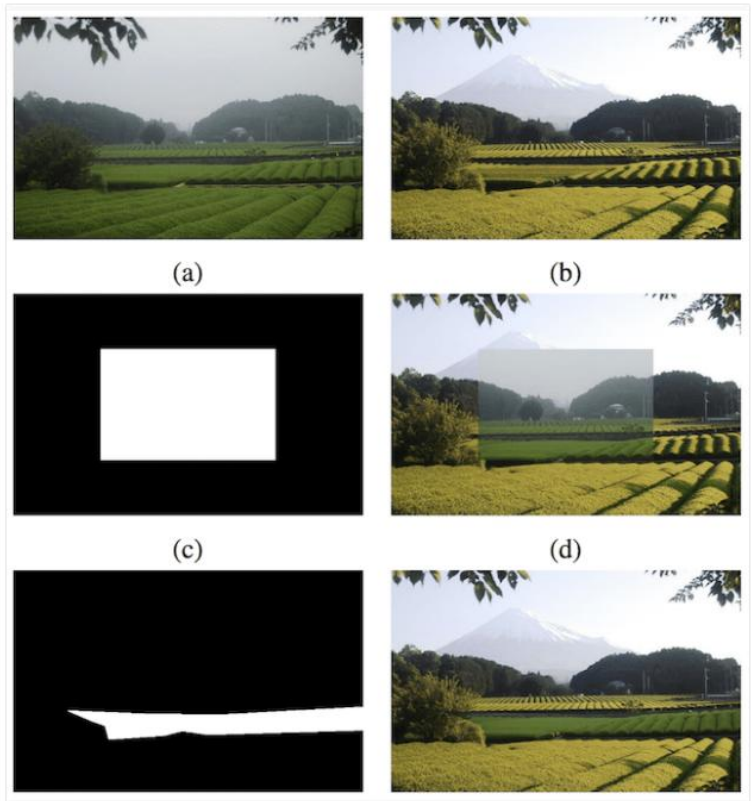
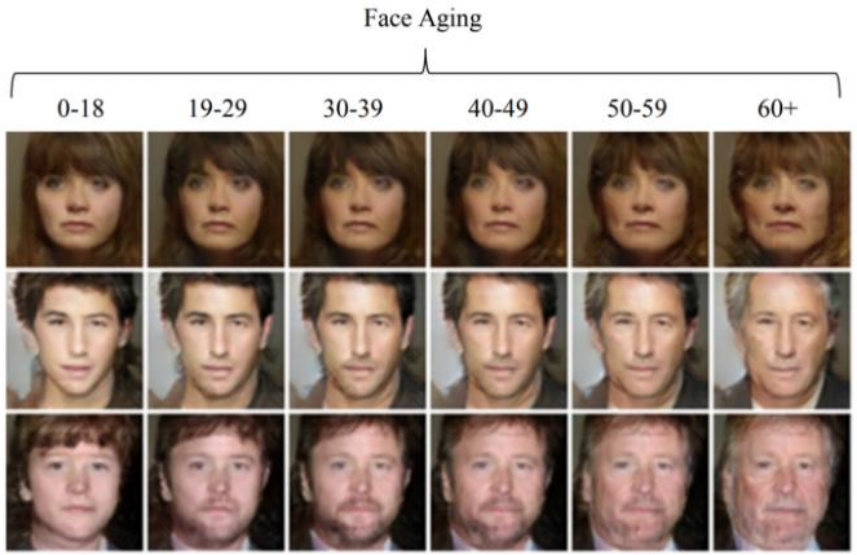
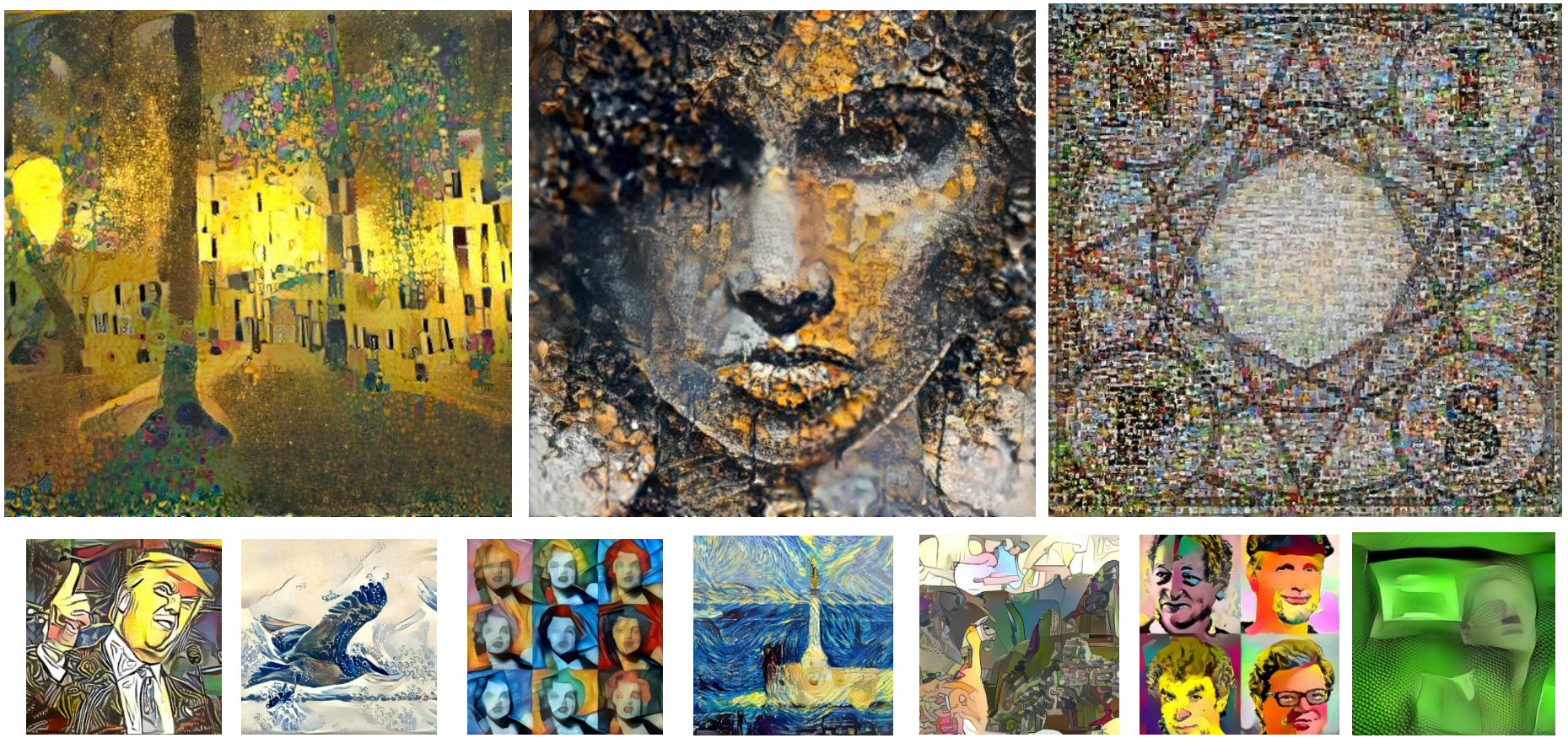


Photo Blending



Face Aging

Generative AI teaser: pre-2022



DeepDream – I created these images around 2016-2017 using two images

2022 – 2023: years of Generative AI







Two minutes Papers channel

Stable Diffusion

. . .

CHAT GPT

DeepDream in 2023



DeepDream – I created these images around 2016-2017 using two images

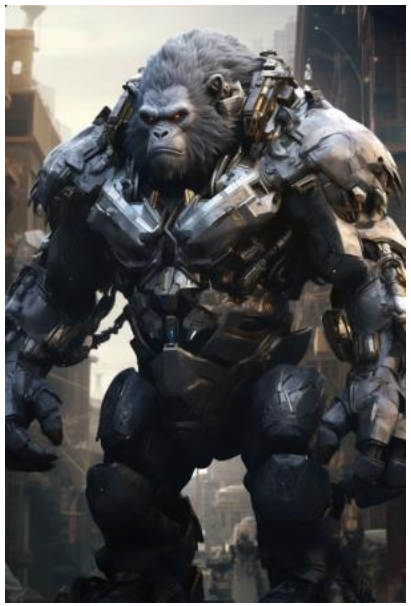




Beyond image generation …



/v5_upscale 13 hrs ago
amazing summer fashion week show runway walking an anthropomorphic female cat as a super model dressed...
Darina



/v5_upscale 22 hrs ago
Armored Core-style gorilla, hyper quality, intricate detail, masterpiece, 8k,
cham38



/upscale 11 months ago
milkyway in a glass bottle, 4K, unreal engine, octane render
Deeds

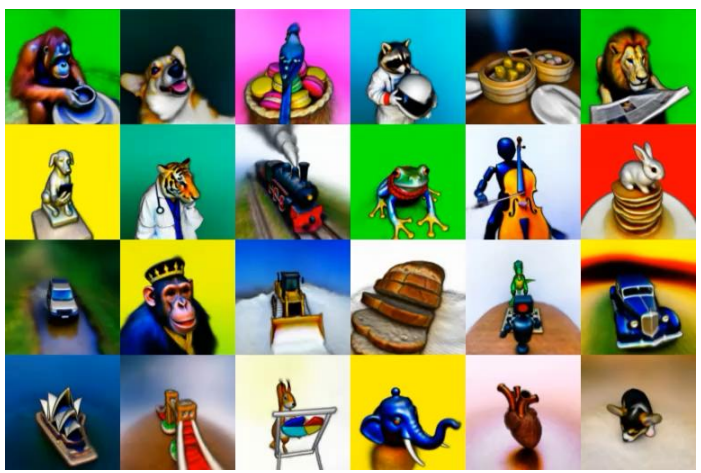


/upscale 10 months ago
portrait photo of a asia old warrior chief, tribal panther make up, blue on red, side profile, looking away, serious...
MantaRay

Image credit: https://www.midjourney.com/showcase

Beyond image generation …

1. 3D generation from a single image: https://3d-diffusion.github.io/
2. Video generation: https://video-diffusion.github.io/
3. Text to 3D: https://dreamfusion3d.github.io/
4. Subject-driven Image generation: https://dreambooth.github.io/
5. Image generation with visual prompt: https://github.com/lllyasviel/ControlNet



Text to 3D



Visual prompt with ControlNet



and of course, ChatGPT and LLMs

ARTIFICIAL INTELLIGENCE

OpenAI's new language generator GPT-3 is shockingly good—and completely mindless

The AI is the largest language model ever created and can generate amazing human-like text on demand but won't bring us closer to true intelligence.

By Will Douglas Heaven July 20, 2020

Meet GPT-3. It Has Learned to Code (and Blog and Argue).

The latest natural-language system generates tweets, pens poetry, summarizes emails, answers trivia questions, translates languages and even writes its own computer programs.



https://businessday.ng/technology/article/chatgpt-is-fastest-app-to-hit-100m-users-in-history/

The current rise of diffusion methods …

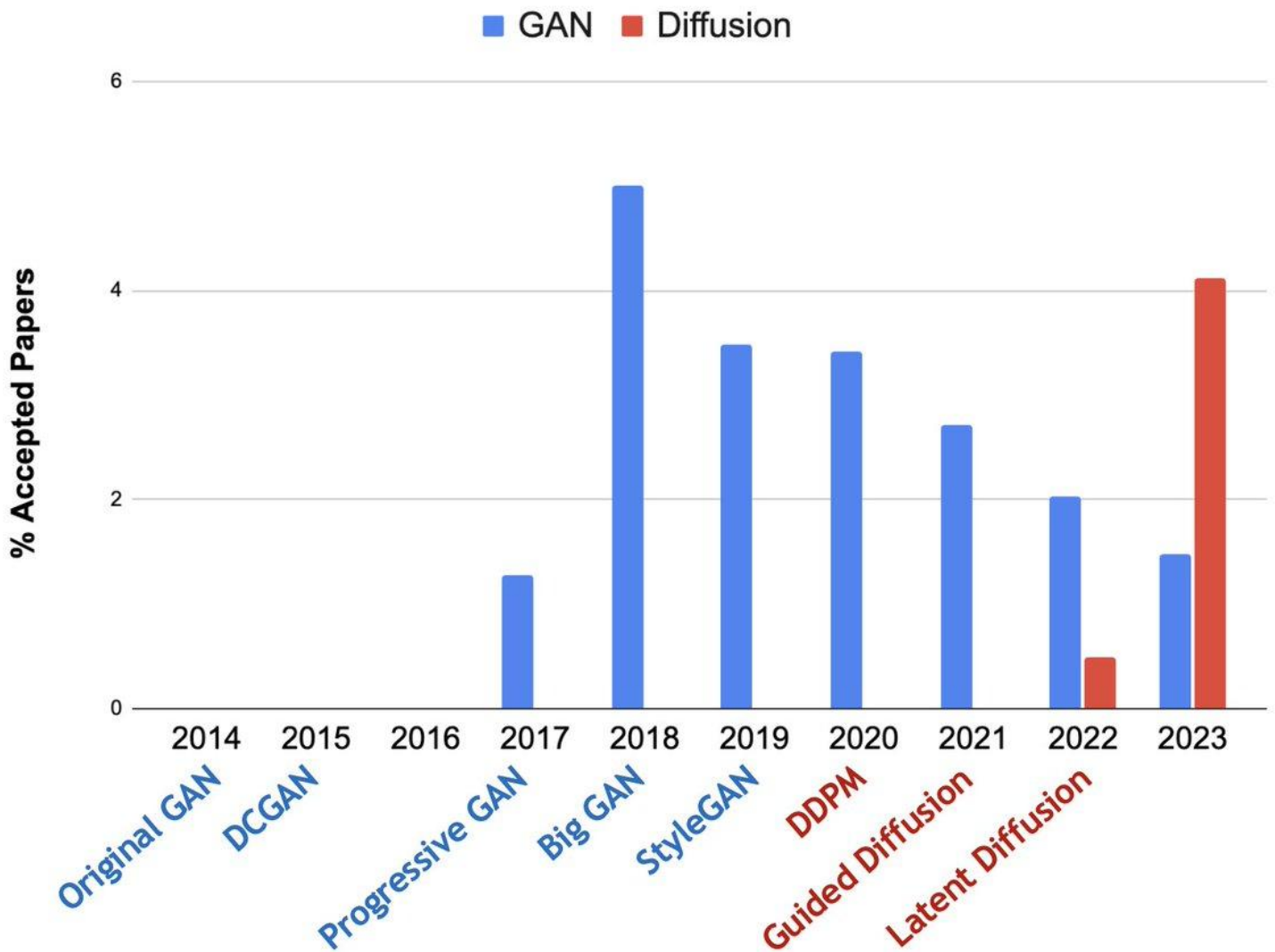


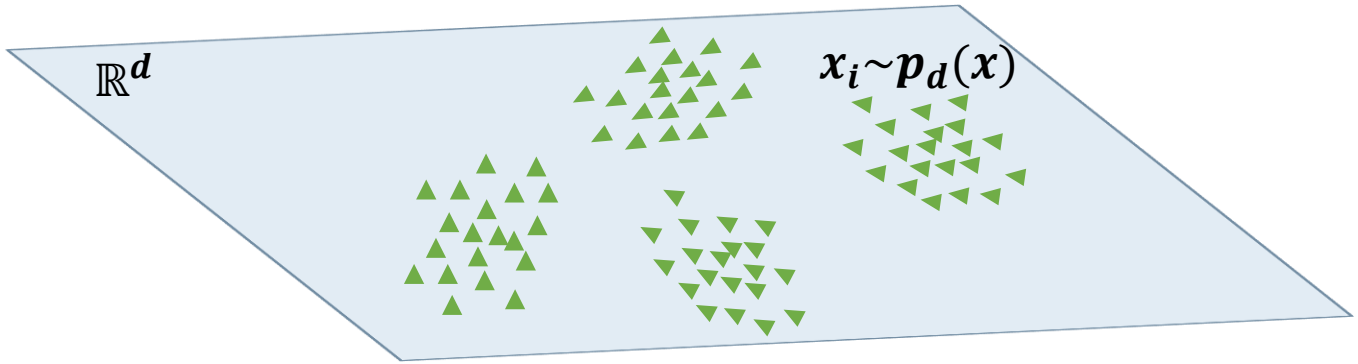
Image credit: https://cvpr2023-tutorial-diffusion-models.github.io/

Outline

- Introduction to generative models
  - Explicit versus implicit density estimation
  - What is a generative model?
  - Why do we need to be generative?
- Generative Adversarial Networks
  - Adversarial training and minimax two-player game
  - Formulation and optimal solution
  - Issues and challenges with GAN
- Diffusion Models

Explicit vs Implicit Density Estimation

Explicit Density Estimation



Dataset $D = \{x_1, x_2, \dots, x_N\}$ where $x_i \in \mathbb{R}^d$
and $x_i \sim p_d(x)$ (data distribution **existed** but **unknown**)

- Given training set $D = \{x_1, x_2, \dots, x_N\}$ where $x_i \sim p_d(x)$
  - $p_d(x)$ is **existed** but **unknown** data distribution.
  - Learn $p_m(x)$ from D such that $p_m(x) \approx p_d(x)$.
- Gaussian Mixture Model
  - $p_m(x) = \sum_{i=1}^{K} \pi_k \, N(x \mid \mu_k, \Sigma_k)$
  - We need to estimate π_k, μ_k, Σ_k for $k = 1, \dots, K$
    - Using **Expectation-Maximization** (EM) approach

Explicit Density Estimation

High-dimensional data



Data manifold

[Source: https://lvdmaaten.github.io/tsne/]

Structural data

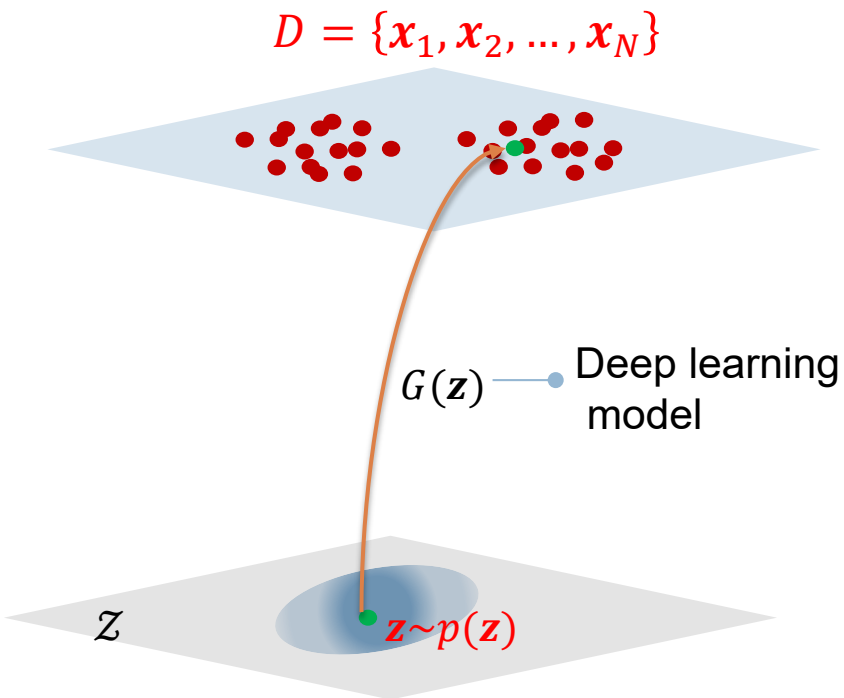


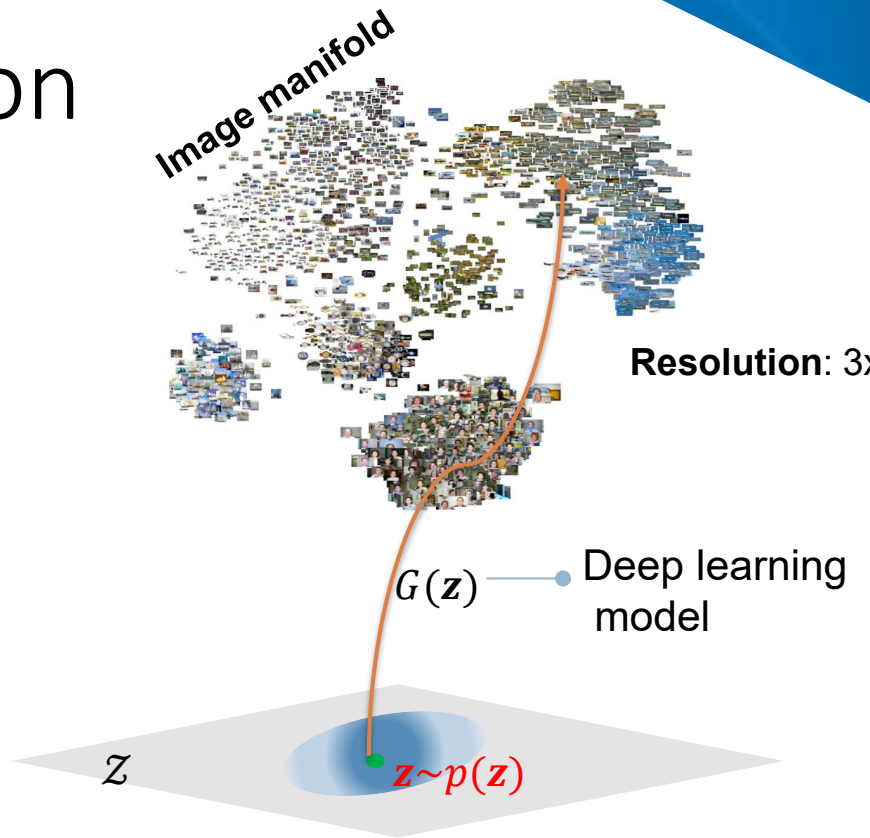




- When does **explicit density estimation** fail?
  - High-dimensional data (curse of dimensionality)
  - Structural data
    - Images, texts, speeches, videos, biological data, and etc.

Implicit Density Estimation

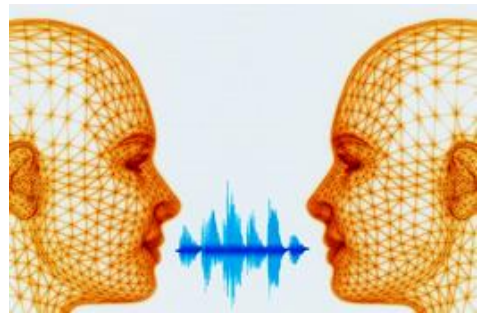


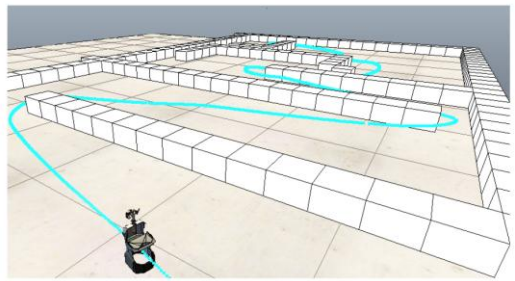


- Given a training set $D = \{x_1, x_2, \ldots, x_N\}$ where each $x_i \sim p_d(x)$.
  - $p_d(x)$ exists but unknown but we do not learn $p_m(x) \approx p_d(x)$
- Learn a **generator G** mapping from the noise space $\mathcal{Z}$ to data space
  - $z \sim p(z) \rightarrow \tilde{x} = G(z) \sim p_d(x)$
  - $\tilde{x} = G(z)$ mimics those in D.
- This is the setting of generative model
  - **Generative Adversarial Network** (GAN), Variational Auto-Encoder (VAE)

Generative models

- **Generative models** learn how data are generated to re-generate new data.
  - Synthesize poem with Markov process
- Texts, speech, videos, interactions
- Simulations
  - Robot learning without damage
  - Learning tasks with few or expensive labels
- Create new art, music, or fashion
- Synthetic biology
- Many more applications …









Generative models

- Explain cause for actions beyond associating inputs to outputs.
  - Explainable AI, patients need to know why they will be treated this way or that way!
- Understand and predict the world - how it evolves!
  - Self-driving needs the ability to predict what happens next on the street
  - Robots need the ability to foresee the environments
  - Fintech AI needs to 'simulate' the future economy, world events, etc.
  - Biologists wish to understand the evolution from data
- Imagine and generate rich plans for the future.
- Recognize objects in the world and their factors of variation.
- Establish concepts useful for reasoning and decision making.
- Detect novelty, surprising events in the world.
- Test our ability to manipulate high-dimensional distributions

[Fellow'14, Mohamed '15, etc]

Generative Models

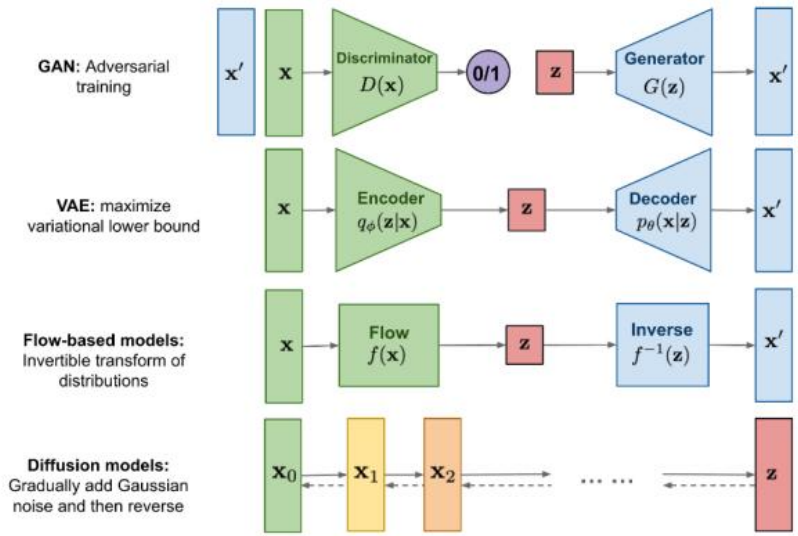
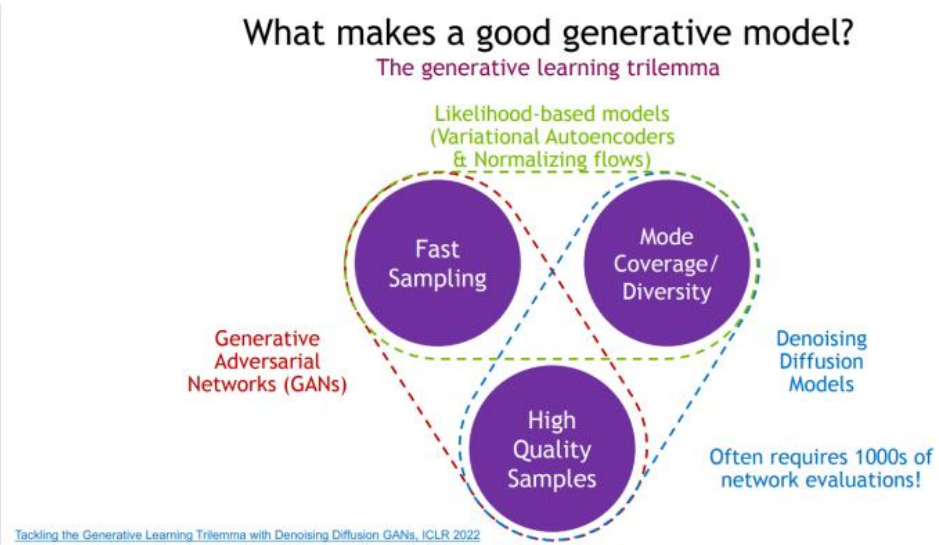


Fig. 1. Overview of different types of generative models.



> The most important one, in my opinion, is adversarial training (also called GAN for Generative Adversarial Networks). This is an idea that was originally proposed by Ian Goodfellow when he was a student with Yoshua Bengio at the University of Montreal (he since moved to Google Brain and recently to OpenAI).
>
> This, and the variations that are now being proposed is the most interesting idea in the last 10 years in ML, in my opinion.



Yann LeCun



Generative Adversarial Networks

> The most important one, in my opinion, is adversarial training (also called GAN for Generative Adversarial Networks). This is an idea that was originally proposed by Ian Goodfellow when he was a student with Yoshua Bengio at the University of Montreal (he since moved to Google Brain and recently to OpenAI).
>
> This, and the variations that are now being proposed is the most interesting idea in the last 10 years in ML, in my opinion.

Yann LeCun



A



B



Which face is NOT generated by AI?

A



B



Which face is NOT generated by AI?

A



B



Which face is NOT generated by AI?

A



B

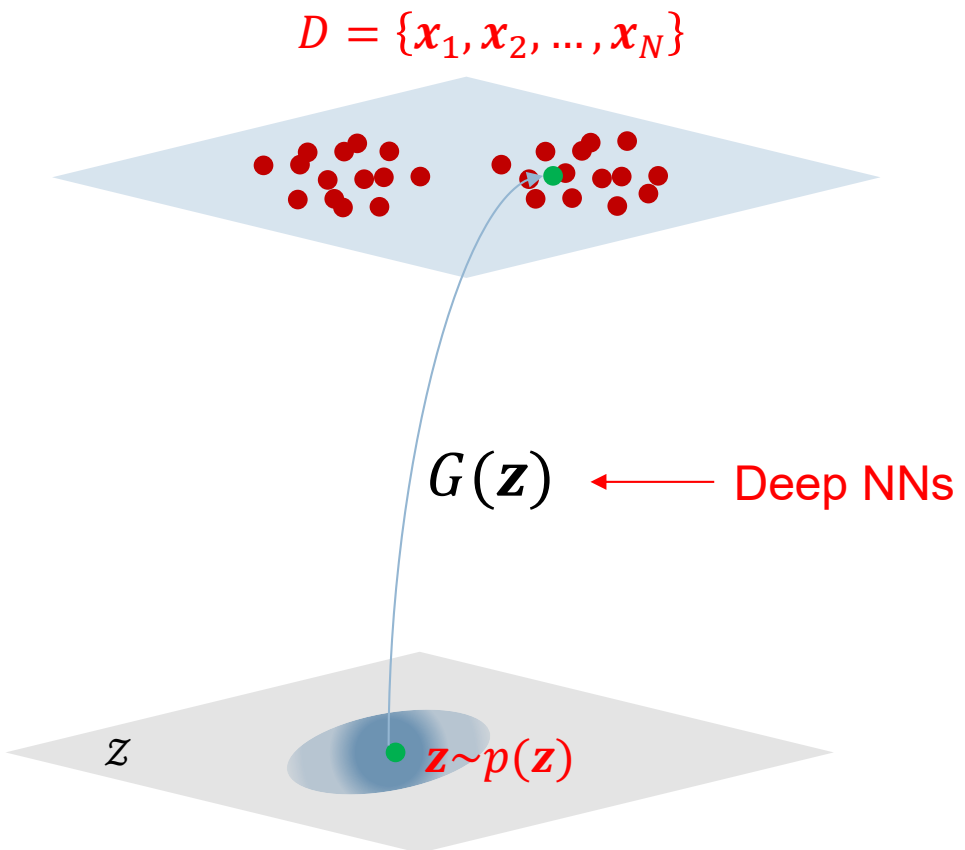


Which face is NOT generated by AI?

http://www.whichfaceisreal.com

Deep Generative Models (DGM)

- Given a training set $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ where each $\boldsymbol{x}_i \sim p_d(\boldsymbol{x})$.
  - $p_d(\boldsymbol{x})$ exists but unknown.
- Learn a **generator $\boldsymbol{G}$** mapping from the noise space $\mathcal{Z}$ to data space
  - $\boldsymbol{z} \sim p(\boldsymbol{z}) \rightarrow \tilde{\boldsymbol{x}} = G(\boldsymbol{z}) \sim p_d(\boldsymbol{x})$
  - $\tilde{\boldsymbol{x}} = G(\boldsymbol{z})$ 'mimics' those in D.
- Two most important and popular families of DGM:
  - Variational Auto Encoder (Kingma et al., 2013)
  - Generative Adversarial Networks (Goodfellow et al., 2014)



Our Task in Generative Models



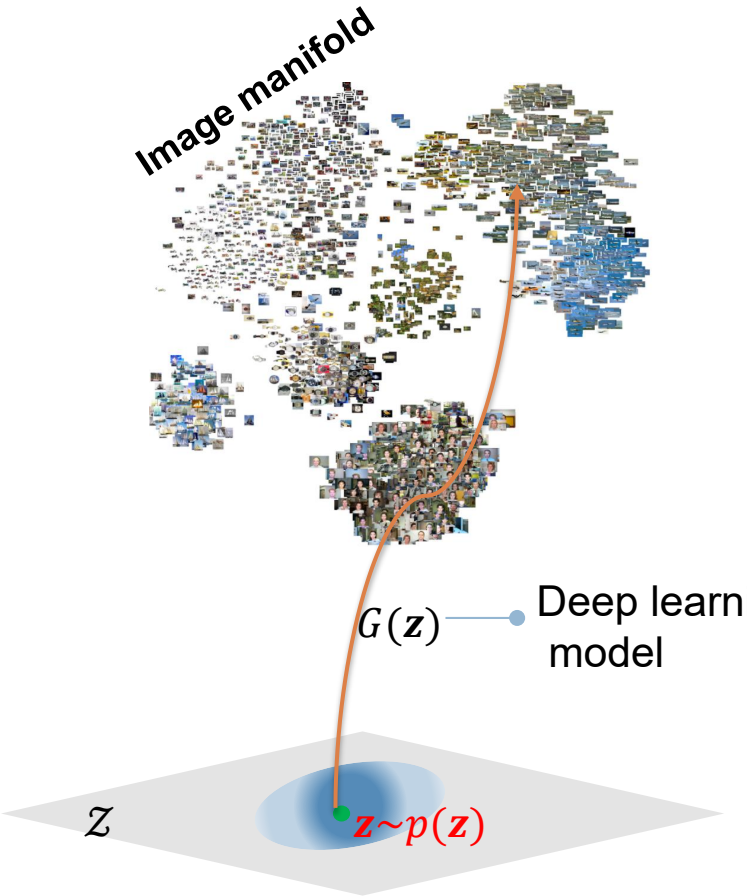
Source of data
(Images, waves, texts)

Train

Trained model (Generator)

Generate



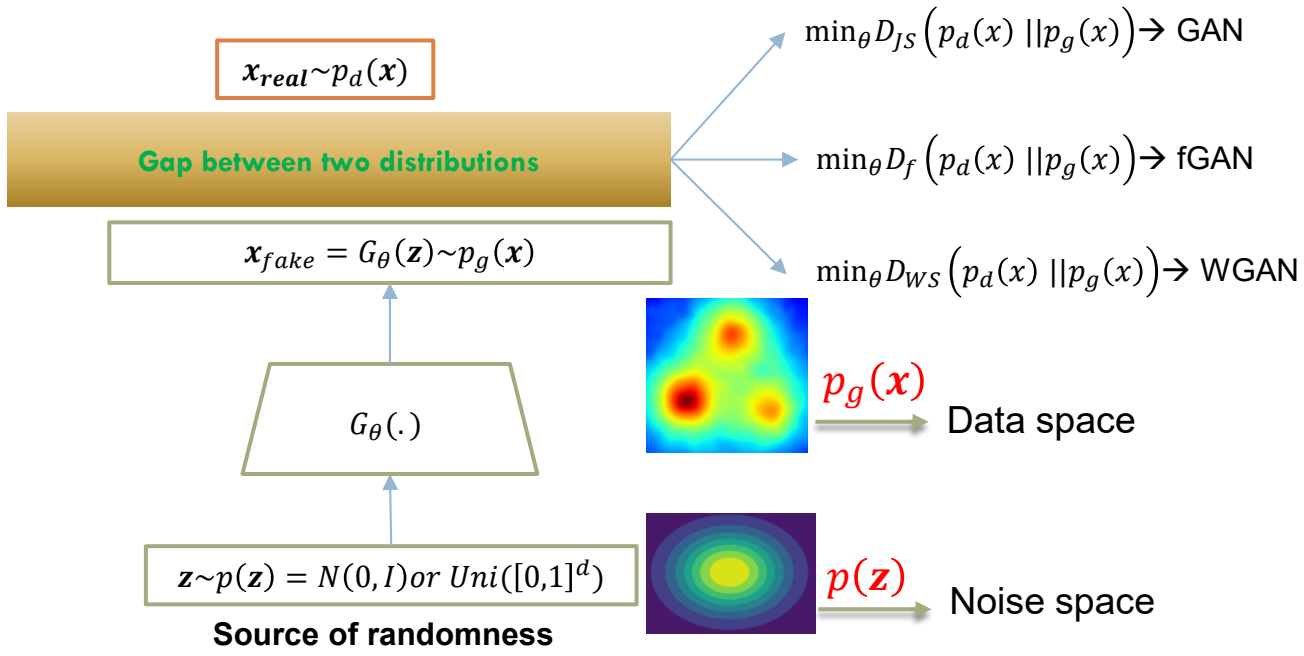


Our Task in Training Generative Models

$D = \{x_1, x_2, \dots, x_N\}$

- $x_i \in \mathbb{R}^d$, $x_i \in \mathbb{R}^{m \times n}$, $x_i = G_i = (V_i, E_i)$
- Assume that x_i(s) are drawn from an **unknown** distribution with p.d.f. $p_d(x)$

- How to use $p_d(\boldsymbol{x})$ since it is **unknown**?
- How to **be aware** of **how far** $p_d(\boldsymbol{x})$ and $p_g(\boldsymbol{x})$?
  - Use a **discriminator** (classifier) to classify the **fake data** and **true data**.
  - If the **accuracy** is **high** and the **loss** is **small**, they are **far away**
  - If the **accuracy** is **low** (50%) and the **loss** is **high**, they are **very close and even overlapping**



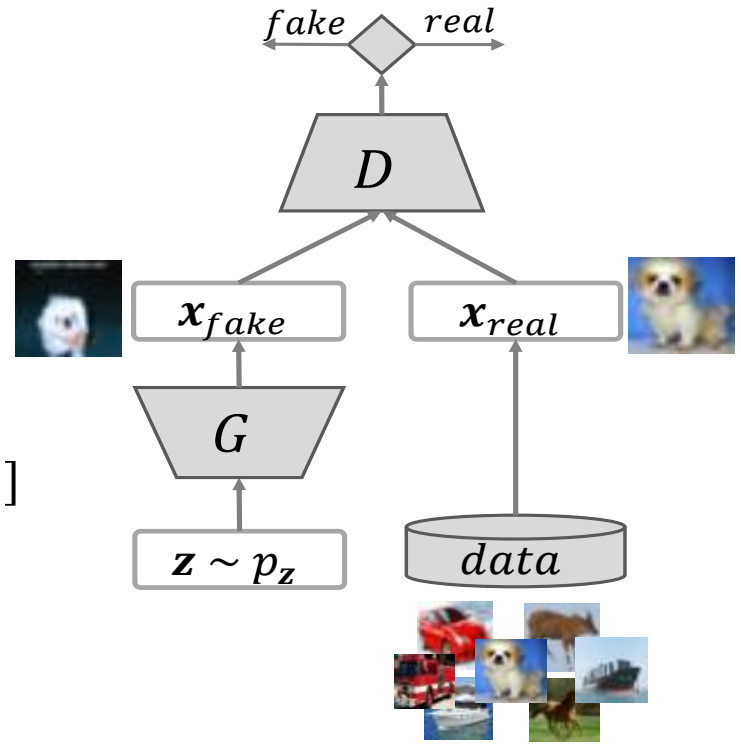
Generative Adversarial Networks

- A game of two players [Goodfellows et al., 2014]
  - **Generator G**: generate fake samples indistinguishable from real samples
  - **Discriminator D**: discriminate real and fake samples

- Min-max problem

$$\min_G \max_D J(G, D) = \mathbb{E}_{\boldsymbol{x} \sim p_d(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z}))]$$

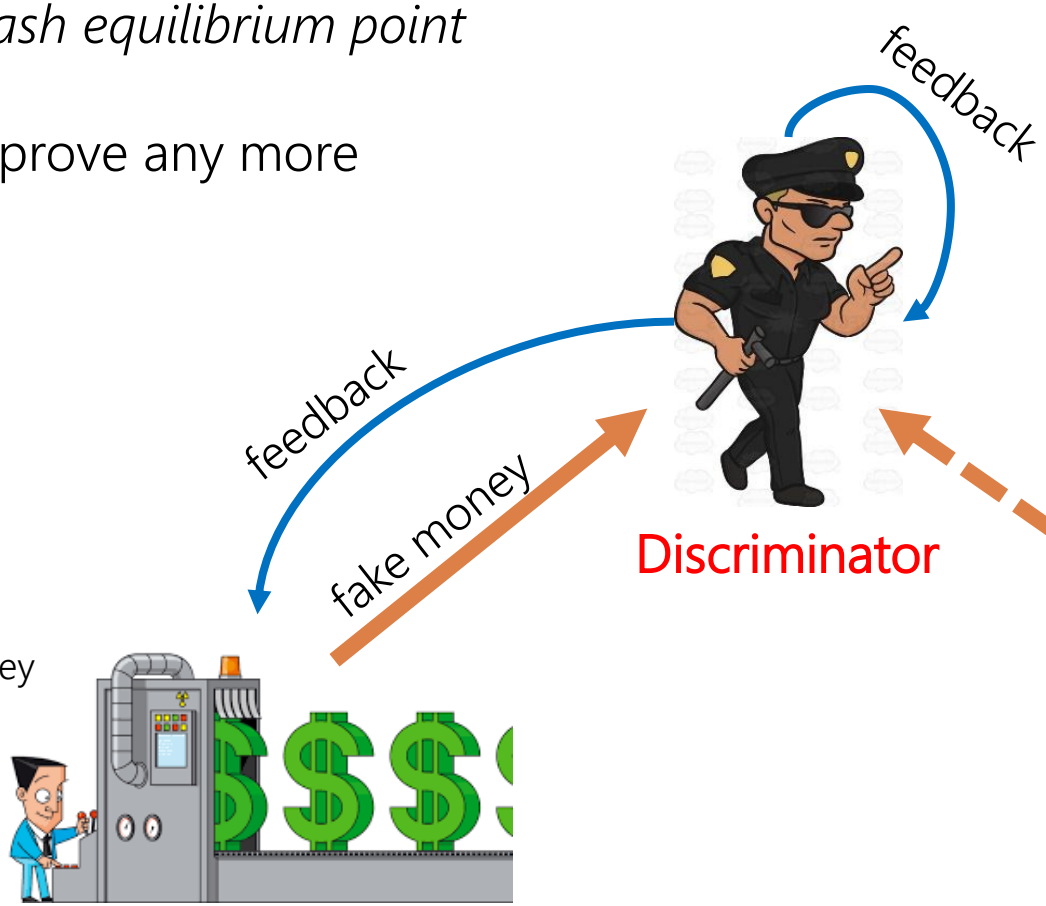
- Issue & challenge
  - Mode collapse, min-max problem



GAN: analogy to minimax game

Termination at *Nash equilibrium point*

No player can improve any more



Task: distinguish between real and counterfeit money

Goal: try to sabotage the counterfeit maker

Task: produce counterfeit money

Goal: try to fool/confuse the police officer



Target/data distribution

Final output from GANs

Generator

Adversarial or Cooperative

Training GAN

- Gradient ascent to train discriminator D_{θ_d}:

$$\max_{\theta_d} \mathop{\mathbb{E}}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}\left[\log D_{\theta_d}(\boldsymbol{x})\right] + \mathop{\mathbb{E}}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[\log(1 - D_{\theta_d}(G(\boldsymbol{z}))\right]$$

Training D is to maximize the probability of detecting correct labels

Ascent update D: $\boldsymbol{z}^{(i)}\sim p_z, \boldsymbol{x}^{(i)}\sim p_{data}$

$$\nabla_{\boldsymbol{\theta}_\mathrm{d}} \frac{1}{\mathrm{M}}\sum_{i=1}^{M}\left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right]$$

$$\min_{G}\max_{\mathrm{D}} J(G, D) = \mathop{\mathbb{E}}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}\left[\log D(\boldsymbol{x})\right] + \mathop{\mathbb{E}}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[\log(1 - D(G(\boldsymbol{z}))\right]$$

Training G is to minimize or fool the discriminator D, i.e., minimize $1 - D\left(G(\boldsymbol{z})\right)$ to 0

Descent update G: $\boldsymbol{z}^{(i)}\sim p_z$

$$-\nabla_{\boldsymbol{\theta}_\mathrm{g}} \frac{1}{\mathrm{M}}\sum_{i=1}^{M}\left[\log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right]$$

- Gradient descent to train the generator G_{θ_g}

$$\min_{\theta_g} \mathop{\mathbb{E}}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[\log(1 - D_{\theta_d}(G(\boldsymbol{z}))\right]$$

GAN algorithm

$k = 1$ is stable

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

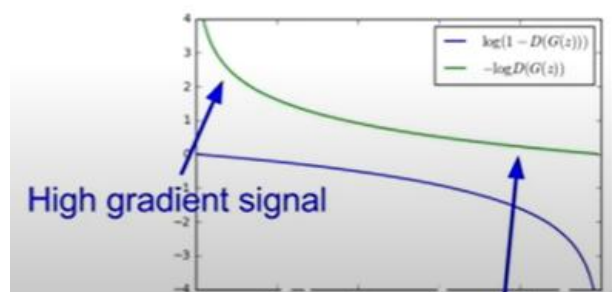
The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

In practice, this objective doesn't work well since gradient signal is dominated by region where the sample looks real, and relatively flat in samples look more fake!

?

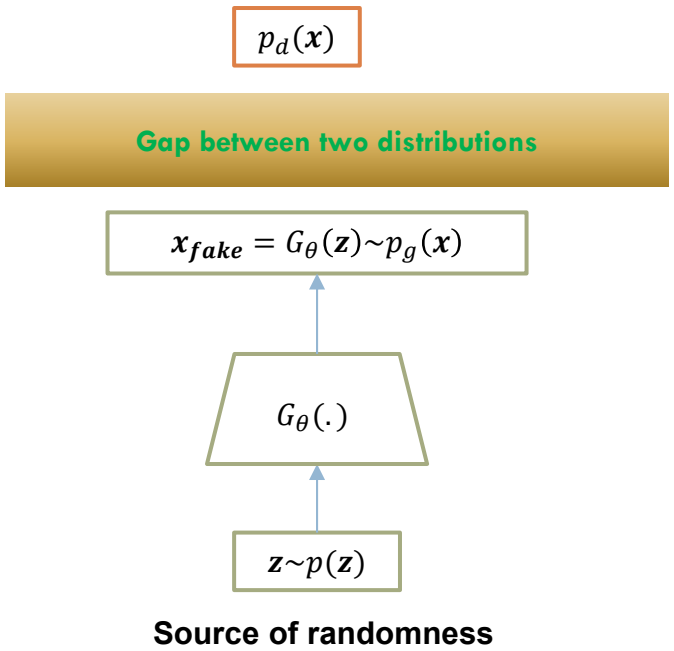
Instead, we can use:

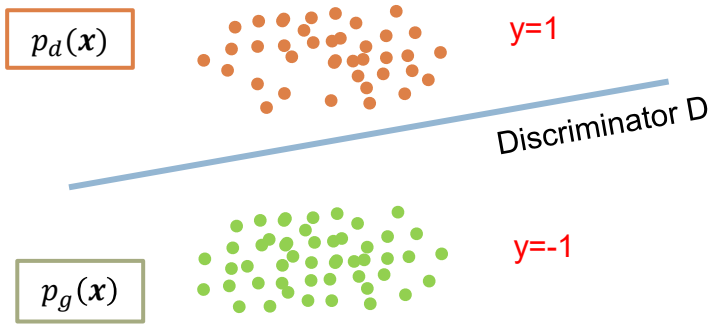
$$\max_{\theta_g} \mathbb{E}_{\boldsymbol{z} \sim p_{\boldsymbol{z}}} \left[\log(D_{\theta_d}(G(\boldsymbol{z})) \right]$$



In general, training GAN using minimax strategy is challenging!

Theory of GAN





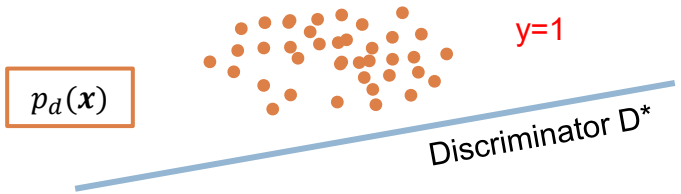
- **D(x) = P(y=1 | x)**: probability **x** is true data.
- **1-D(x) = P(y=-1 | x)**: probability x is fake data.

The **log likelihood function** need to be maximized

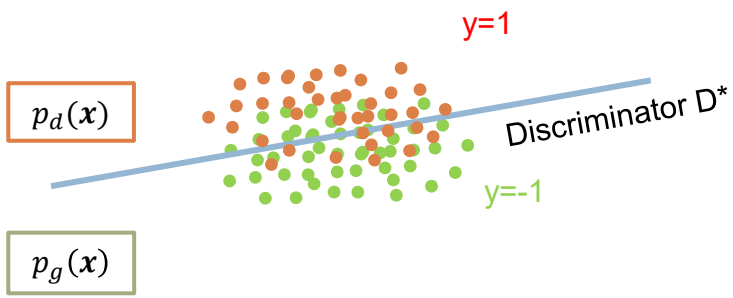
$$J(D,G) = \mathbb{E}_{x \sim P_d}[\log D(x)] + \mathbb{E}_{\tilde{x} \sim P_g}[\log[1 - D(\tilde{x})]]$$
$$= \mathbb{E}_{x \sim P_d}[\log D(x)] + \mathbb{E}_{z \sim P_z}[\log[1 - D(G(z))]]$$

- We want to train **NN-based function** $G_\theta(.)$ such that $p_g(x) = p_d(x)$, but we do not have $p_d(x)$ in hand
  - Invoke a strong discriminator $D(x)$ to **implicitly justify** how far $p_d(x)$ and $p_g(x)$ are.

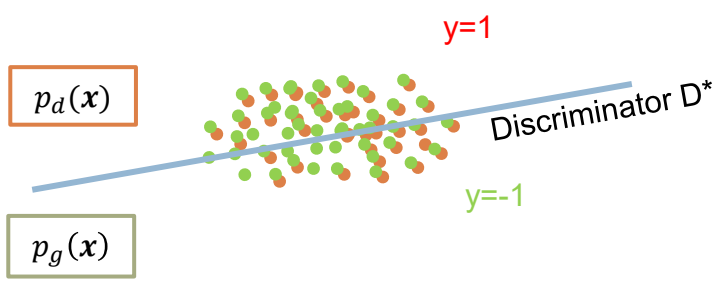
Theory of GAN



$J(D^*, G) = \mathbb{E}_{x \sim P_d}[\log D^*(x)] + \mathbb{E}_{\tilde{x} \sim P_g}[\log[1 - D^*(\tilde{x})]]$ is large



$J(D^*, G) = \mathbb{E}_{x \sim P_d}[\log D^*(x)] + \mathbb{E}_{\tilde{x} \sim P_g}[\log[1 - D^*(\tilde{x})]]$ decreases

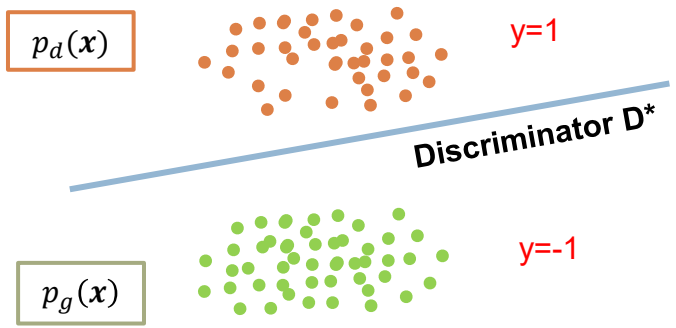


$J(D^*, G) = \mathbb{E}_{x \sim P_d}[\log D^*(x)] + \mathbb{E}_{\tilde{x} \sim P_g}[\log[1 - D^*(\tilde{x})]]$ is smallest

It comes with the **minimax** problem

$$\boldsymbol{min_G max_D J(D, G)}$$

Optimal discriminator



$$J(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_d}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x}\sim p_g}[\log[1 - D(\boldsymbol{x})]]$$
$$= \mathbb{E}_{\boldsymbol{x}\sim p_d}[\log D(\boldsymbol{x})] + \mathbb{E}_{z\sim p_z}[\log[1 - D(G(\boldsymbol{z}))]]$$

- $\boldsymbol{min_G max_D J(G,D)}$
  - Given **G**, we need to solve:
    $\boldsymbol{D^* = D^*(G) = argmax_D J(D,G)}$

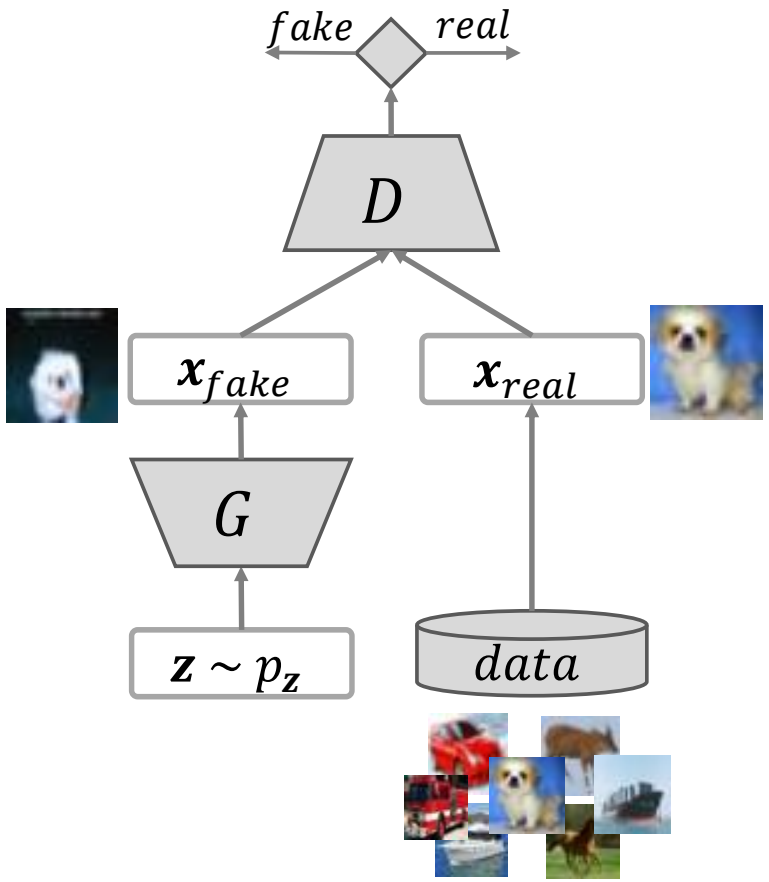
- The **optimal discriminator** has the following form:
  - $D^*(\boldsymbol{x}) = \frac{p_d(\boldsymbol{x})}{p_d(\boldsymbol{x}) + p_g(\boldsymbol{x})}, for\ all\ \boldsymbol{x}$

Optimal discriminator

$$J(D,G) = \mathbb{E}_{\boldsymbol{x}\sim P_d}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x}\sim P_g}[\log[1 - D(\boldsymbol{x})]]$$

$$= \int [p_d(\boldsymbol{x}) \log D(\boldsymbol{x}) + p_g(\boldsymbol{x}) log(1 - D(\boldsymbol{x}))] d\boldsymbol{x}$$

- We need to solve:
  - $max_D\left(\int [p_d(\boldsymbol{x}) \log D(\boldsymbol{x}) + p_g(\boldsymbol{x}) log(1 - D(\boldsymbol{x}))] d\boldsymbol{x}\right)$
- We need to maximize D, let's do this pointwise at each position of x:
  - $max_D[h(D) := p_d(\boldsymbol{x}) \log D + p_g(\boldsymbol{x}) \log(1 - D)]$
  - $\nabla h(D) = \frac{p_d(x)}{D} - \frac{p_g(x)}{1-D} = 0$
  - It means that $D^*(\boldsymbol{x}) = \frac{p_d(x)}{p_d(x)+p_g(x)}$



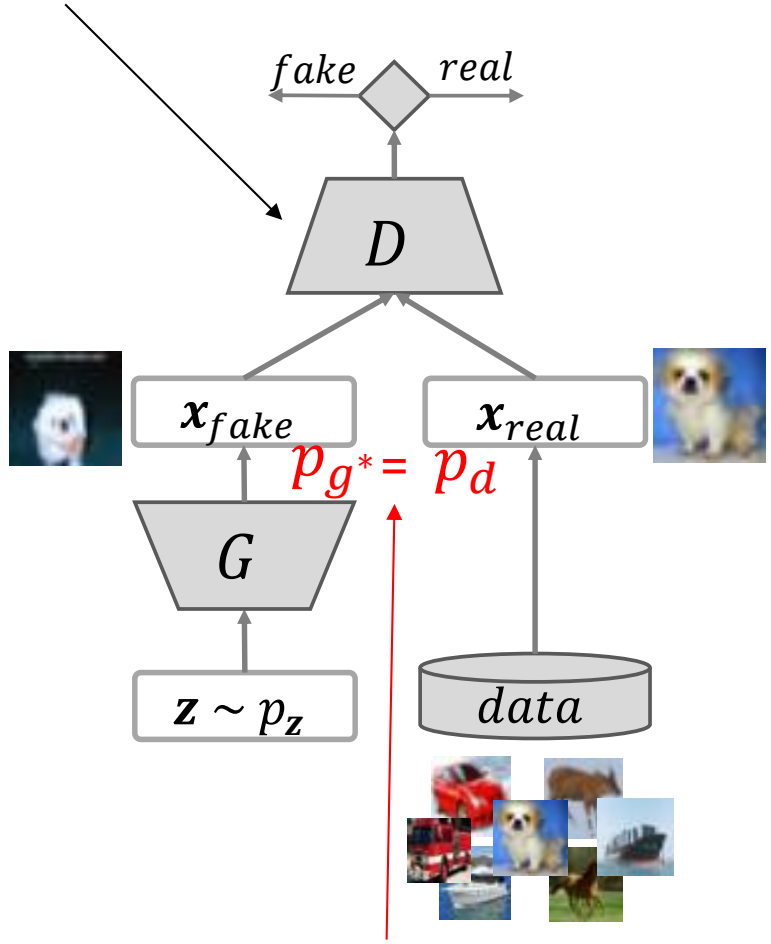
Optimal generator

- $\boldsymbol{min_G max_D J(D,G) = min_G J(D^*(G), G)}$
- $J(D^*(G), G) = \mathbb{E}_{\boldsymbol{x}\sim p_d}[\log D^*(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x}\sim p_g}[\log[1 - D^*(\boldsymbol{x})]] =$

  $\mathbb{E}_{\boldsymbol{x}\sim p_d}\left[\log \frac{p_d(\boldsymbol{x})}{p_d(\boldsymbol{x})+p_g(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_g}\left[\log\left[1 - \frac{p_d(\boldsymbol{x})}{p_d(\boldsymbol{x})+p_g(\boldsymbol{x})}\right]\right] = \mathbb{E}_{\boldsymbol{x}\sim p_d}\left[\log \frac{p_d(\boldsymbol{x})}{p_d(\boldsymbol{x})+p_g(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_g}\left[\log \frac{p_g(\boldsymbol{x})}{p_d(\boldsymbol{x})+p_g(\boldsymbol{x})}\right] =$

  $D_{JS}^{0.5,0.5}\left(p_d(\boldsymbol{x})||p_g(\boldsymbol{x})\right) - 2\log 2$
- We come with
  - $min_G\left(D_{JS}^{0.5,0.5}\left(p_d(\boldsymbol{x})||p_g(\boldsymbol{x})\right)\right) \rightarrow p_{g^*} = p_d$

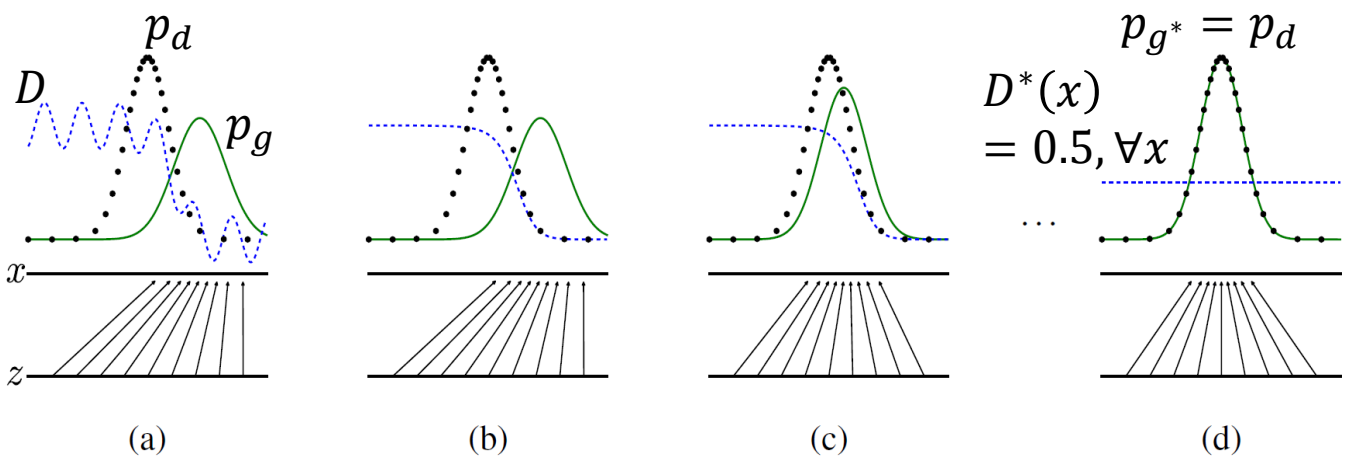
When training is perfect, discriminator D can NO longer tell the fake (money) from the real (money), i.e., 0.5 probability



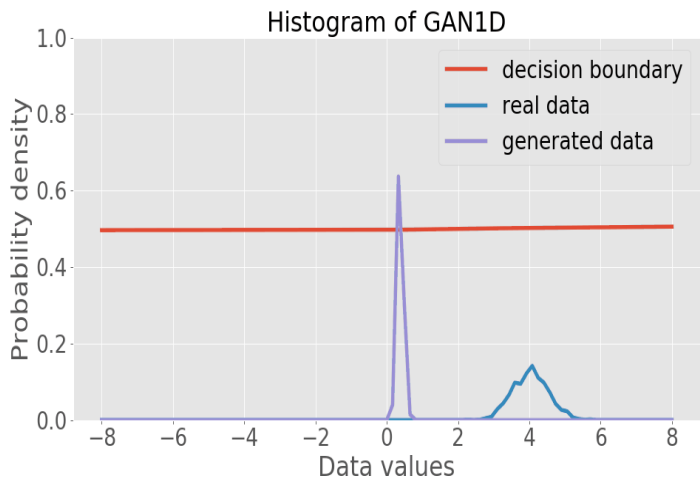
Optimal G

The Nash equilibrium point

- The solution of the minimax problem $\boldsymbol{min_G max_D J(D, G)}$
  - Nash equilibrium point (D^*, G^*) which satisfies
    - $p_{g^*} = p_d$
    - $D^*(\boldsymbol{x}) = \frac{p_d(\boldsymbol{x})}{p_d(\boldsymbol{x}) + p_{g^*}(\boldsymbol{x})} = 0.5, for\ all\ x$



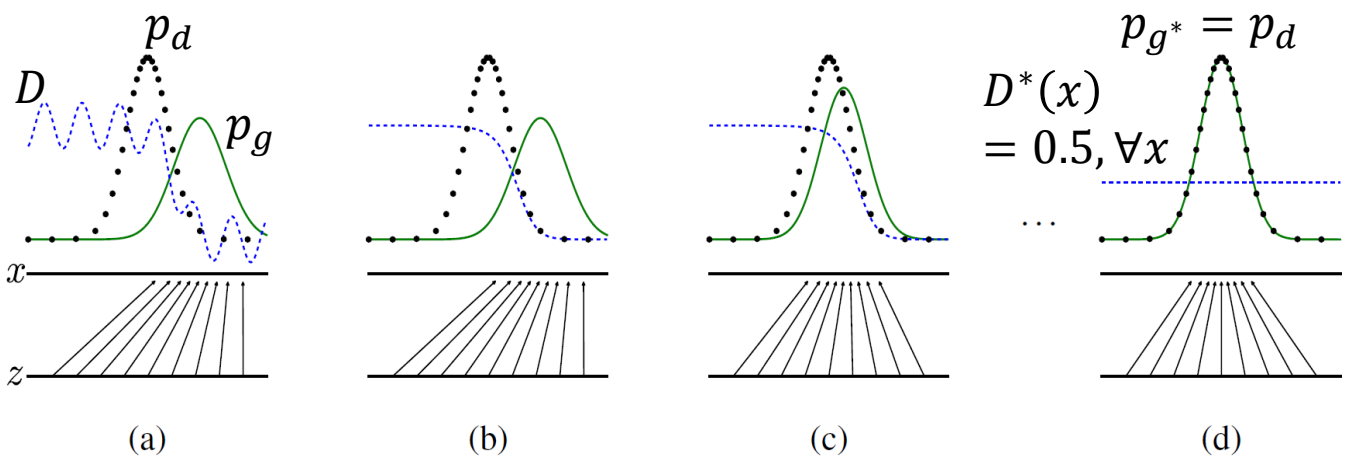
[Goodfellow et al, NIPS 14]



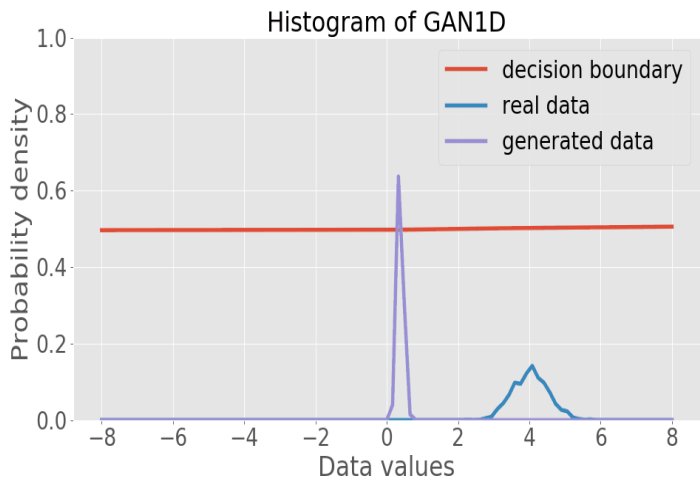
Our own simulation

The Nash equilibrium point

- The solution of the minimax problem $\boldsymbol{min_G max_D J(D, G)}$
  - Nash equilibrium point (D^*, G^*) which satisfies
    - $p_{g^*} = p_d$
    - $D^*(\boldsymbol{x}) = \frac{p_d(\boldsymbol{x})}{p_d(\boldsymbol{x}) + p_{g^*}(\boldsymbol{x})} = 0.5, for\ all\ x$

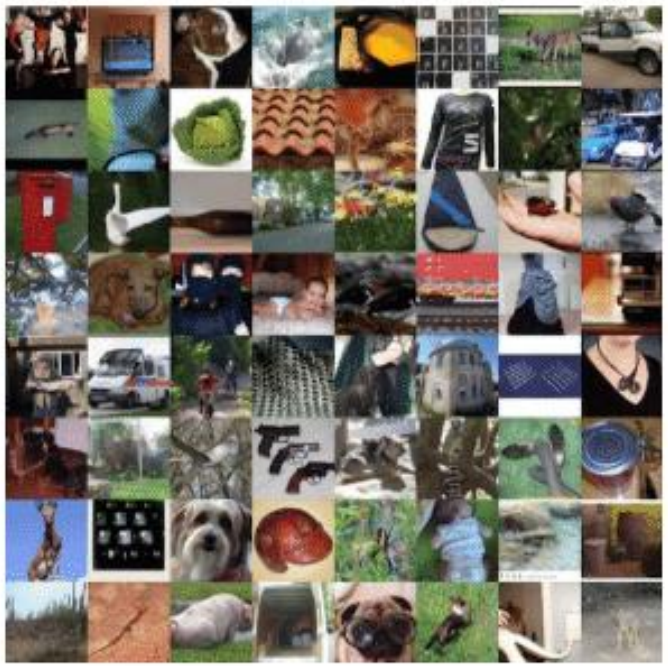


[Goodfellow et al, NIPS 14]



Our own simulation

GAN: example results



Real images (ImageNet)



Generated images

- For a complex dataset such as ImageNet
  - GANs can generate sharp images, but look unrealistic
  - Hard to know what objects they are
- Conditional GANs with labels work much better in practice!

Issues with GAN

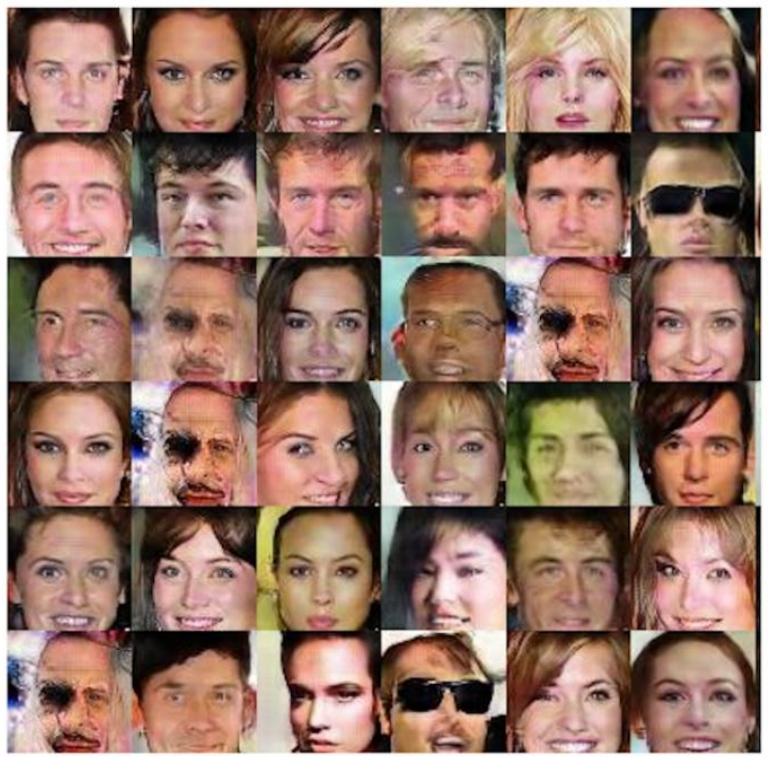
Two serious technical problems with GAN

- #1: mode collapsing problem
  - Impeach the ability to generate diverse, realistic data/images



Mode collapse example 1: Generate only some digits {0, 1, 3, 5, 7} while learning from MNIST dataset

- Generated (fake) examples $G(\boldsymbol{z})$ where $\boldsymbol{z} \sim p(\boldsymbol{z})$
  - Can only cover a **few modes** in the data distribution $p_d(\boldsymbol{x})$
  - Miss **many other modes** in the data distribution.

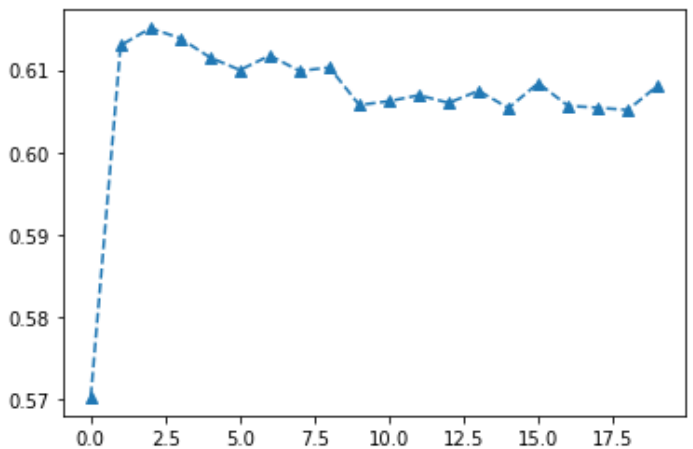


Mode collapse example 2: Generate only some faces while learning from CelebA dataset

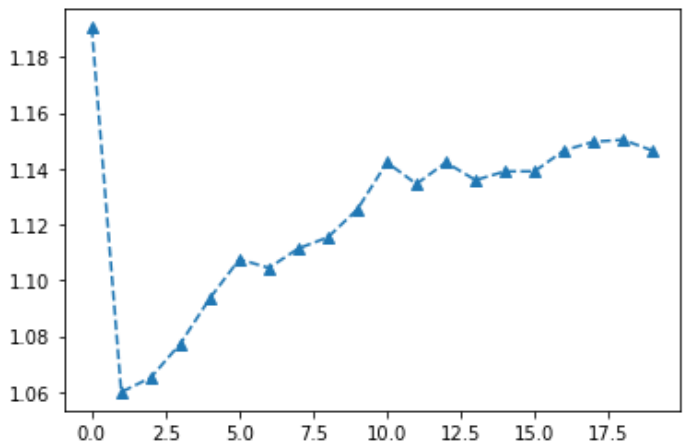
Issues with GAN

Two serious technical problems with GAN

- #2: Convergence is hard due to minimax formulation
  - Training GAN is very challenging!



Discriminator loss

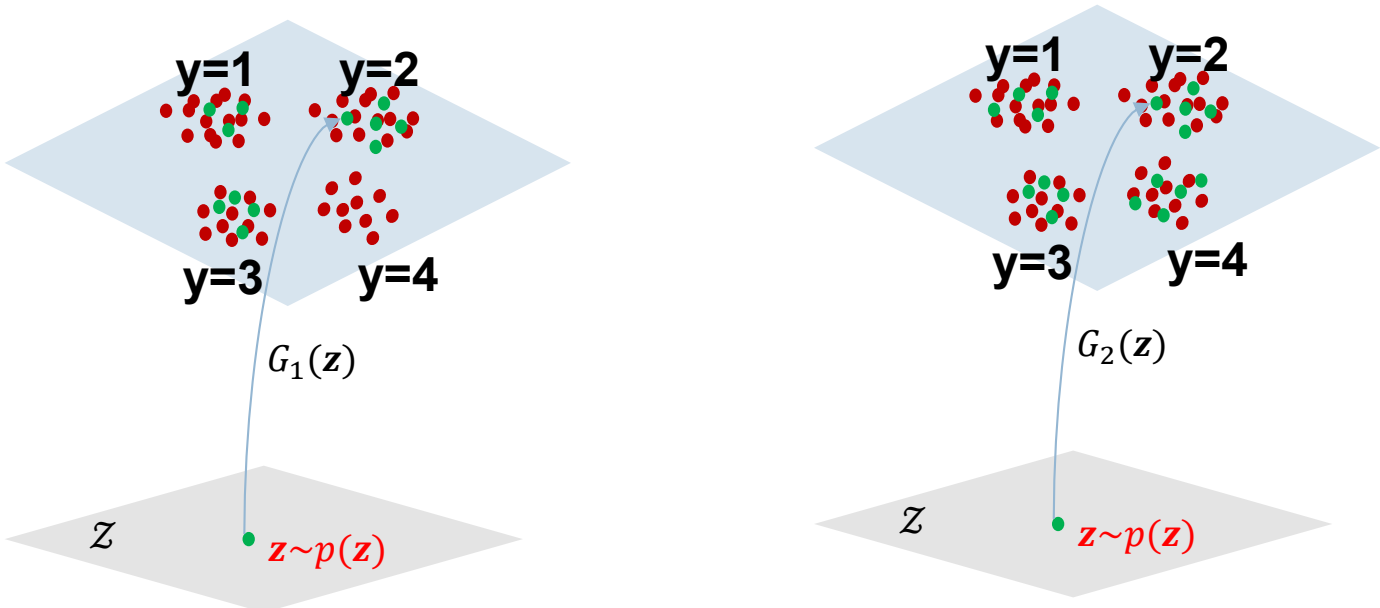


Generator loss

- Mini-max problem
  - $\min_G \max_D J(G, D) = \mathbb{E}_{x \sim p_d(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z))]$
- No unique loss function

How to evaluate GAN: Inception score

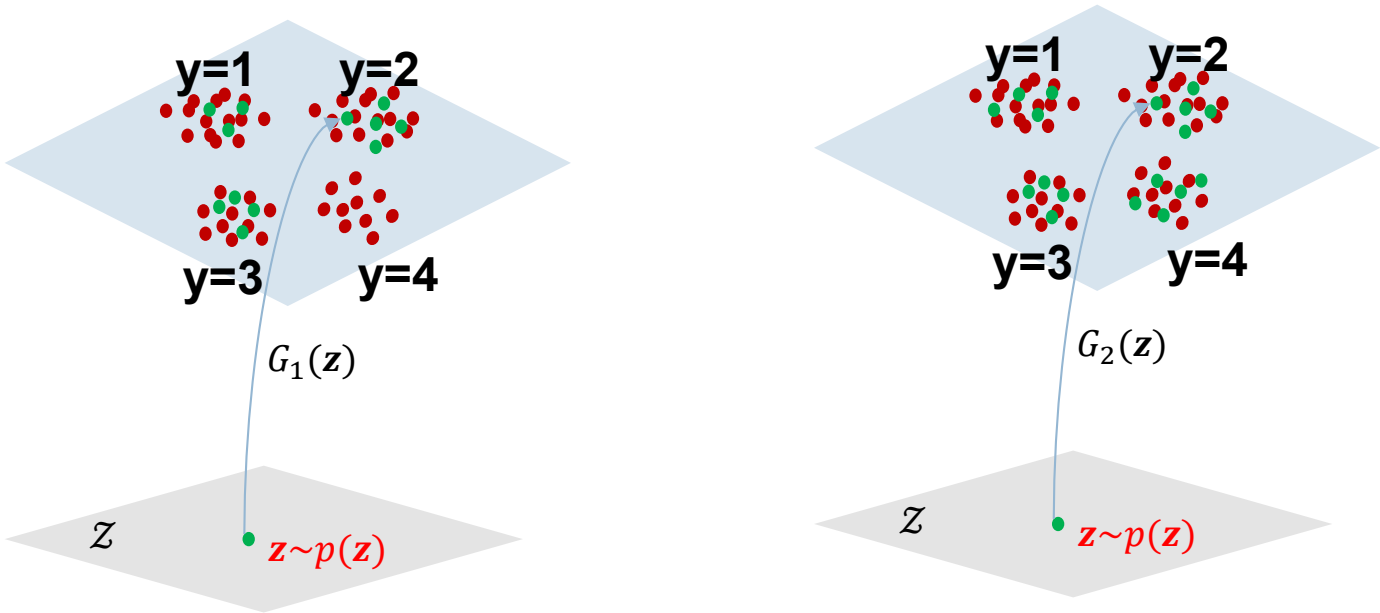
- Except for trivial cases, we don't know the true data distribution p_d so how can we say how good the generated distribution p_g is?
- How to know that a generative model can generate **diverge examples** that cover **many modes** in data?
- How to **compare two generative models** trained on the same training set?



Two generators $\boldsymbol{G_1}$ and $\boldsymbol{G_2}$, which one is better?

How to evaluate GAN: Inception score

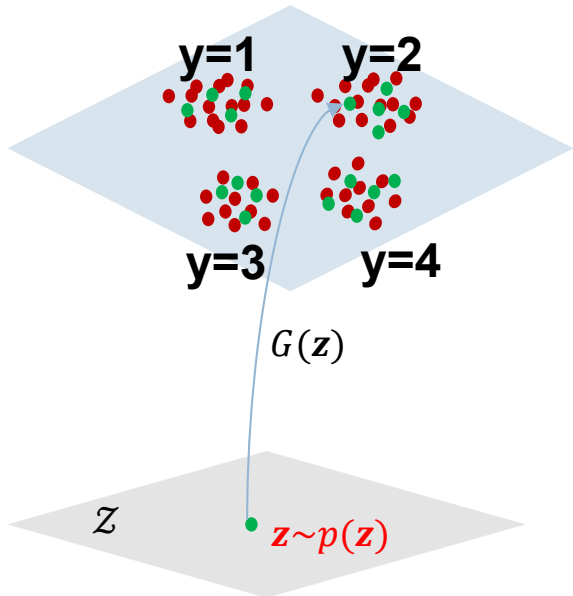
- Except for trivial cases, we don't know the true data distribution p_{data} so how can we say how good the generated distribution p_g is?
- How to know that a generative model can generate **diverge examples** that cover **many modes** in data?
- How to **compare two generative models** trained on the same training set?



G_2 is better and generate more diverge examples, while G_1 misses the data mode y=4.

How to evaluate GAN: Inception score

- Assume that training examples have labels
  - y=1,2,3, and 4.
- We train a good classifier C on the training set with labels
  - $C(\boldsymbol{x}) = p(y \mid \boldsymbol{x}) = [p(y = k \mid \boldsymbol{x})]_{k=1}^{4}$ is the prediction probabilities where $p(y = k \mid \boldsymbol{x})$ is the probability to classify x to the class k.
- Given $\tilde{\boldsymbol{x}}_i = G(\boldsymbol{z_i}), i = 1, \dots, T$, we wish
  - $C(\tilde{\boldsymbol{x}}_i) = p(y \mid \tilde{\boldsymbol{x}}_i = G(\boldsymbol{z_i}))$ is **close** to an **one-hot vector**
    - $C(\tilde{\boldsymbol{x}}_i)$ is **confident** and has a **small entropy**.
  - $\frac{1}{T}\sum_{i=1}^{T} C(\tilde{\boldsymbol{x}}_i) = \frac{1}{T}\sum_{i=1}^{T} p(y \mid \tilde{\boldsymbol{x}}_i = G(\boldsymbol{z_i})$ is close to the **uniform distribution** $[0.25, 0.25, 0.25, 0.25]$ (big entropy).
- Inception score
  - $\frac{1}{T}\sum_{i=1}^{T} KL\left(C(\tilde{\boldsymbol{x}}_i), \frac{1}{T}\sum_{j=1}^{T} C(\tilde{\boldsymbol{x}}_j)\right) \approx \mathbb{E}_z[KL(p(y \mid \tilde{\boldsymbol{x}} = G(\boldsymbol{z})), p(y))]$
  - **Higher** is **better** → **more diversity** of generated images



Salimans et al, *Improved Techniques for Training GANs*, 2016

GAN: Summary

- Adversarial training: a game of two players [Goodfellows et al., 2014]
  - Generator G: generate fake samples indistinguishable from real samples
  - Discriminator D: discriminate real and fake samples
- Min-max problem

$$\min_G \max_D J(G, D) = \mathbb{E}_{\boldsymbol{x} \sim p_d(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z}))]$$

- Issues and challenges
  - Mode collapse, min-max problem

> The most important one, in my opinion, is adversarial training (also called GAN for Generative Adversarial Networks). This is an idea that was originally proposed by Ian Goodfellow when he was a student with Yoshua Bengio at the University of Montreal (he since moved to Google Brain and recently to OpenAI).
>
> This, and the variations that are now being proposed is the most interesting idea in the last 10 years in ML, in my opinion.

Yann Lecun



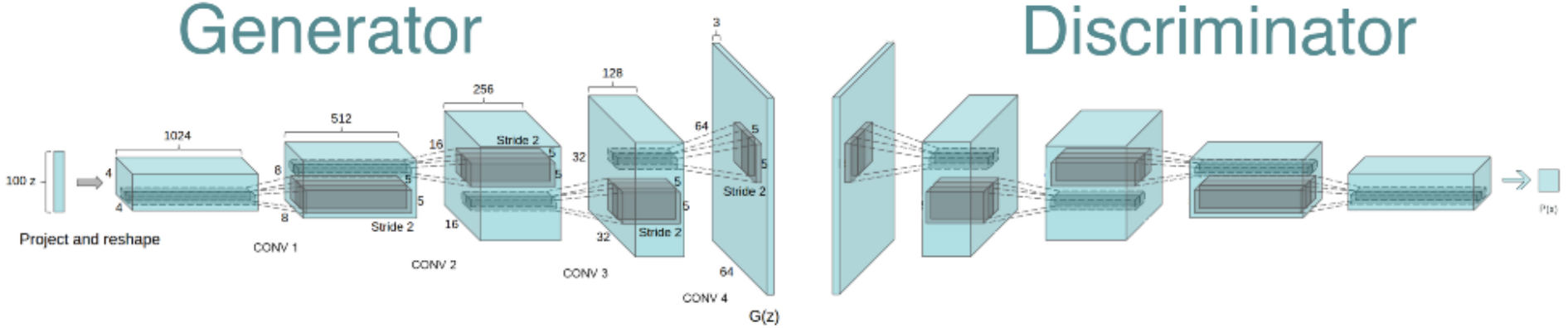

Q&A
WHO?
WHERE?
WHEN?
WHAT?
HOW?
Why?
When?
Where?
What?

Deep Convolutional GAN (DCGAN)

Radford et al, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”, 2015

Introducing Conv2DTranspose

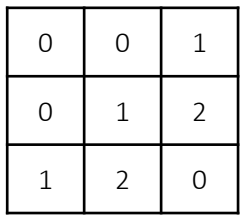
- Why do we need the **Conv2DTranspose** layer?



DCGAN architecture

Introducing Conv2DTranspose

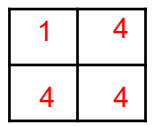
- Standard Conv2D
  - **Conv2D:** input = [3,3], kernel = [2,2], stride = [1,1] → output = [2,2]



input

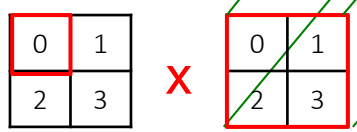


filter

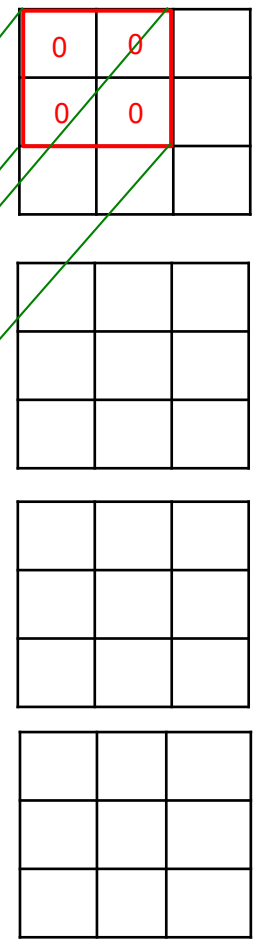


output

- Conv2Dtranpose: it is <u>opposite</u> of conv2D
  - Input =[2,2], kernel = [2,2], stride = [1,1] → output = [3,3]

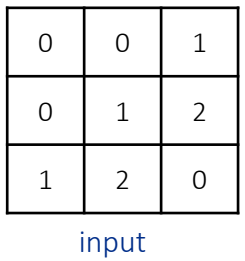


input filter



Introducing Conv2DTranspose

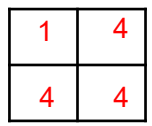
- Standard Conv2D
  - **Conv2D:** input = [3,3], kernel = [2,2], stride = [1,1] → output = [2,2]



input

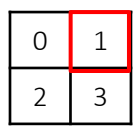


filter



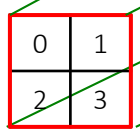
output

- Conv2Dtranpose: it is <u>opposite</u> of conv2D
  - Input =[2,2], kernel = [2,2], stride = [1,1] → output = [3,3]

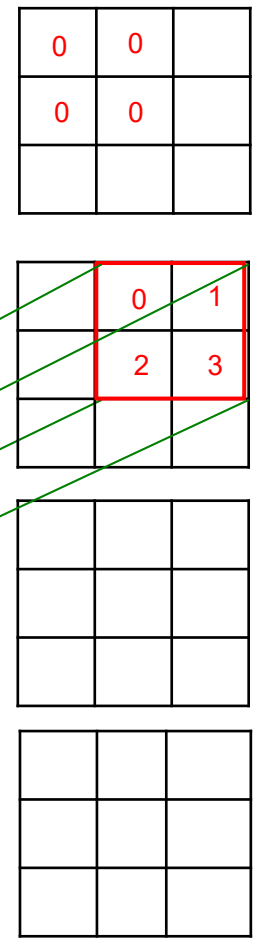


input

X

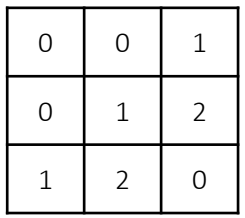


filter



Introducing Conv2DTranspose

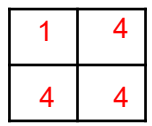
- Standard Conv2D
  - **Conv2D:** input = [3,3], kernel = [2,2], stride = [1,1] → output = [2,2]



input

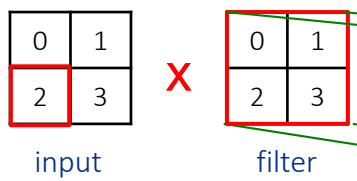


filter

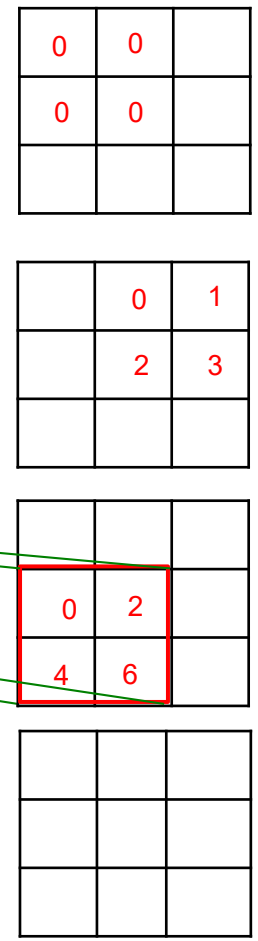


output

- Conv2Dtranpose: it is <u>opposite</u> of conv2D
  - Input =[2,2], kernel = [2,2], stride = [1,1] → output = [3,3]

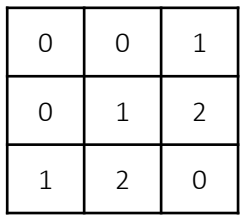


input filter



Introducing Conv2DTranspose

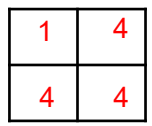
- Standard Conv2D
  - **Conv2D:** input = [3,3], kernel = [2,2], stride = [1,1] → output = [2,2]



input

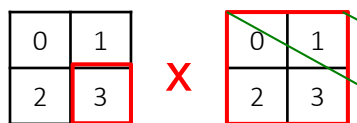


filter



output

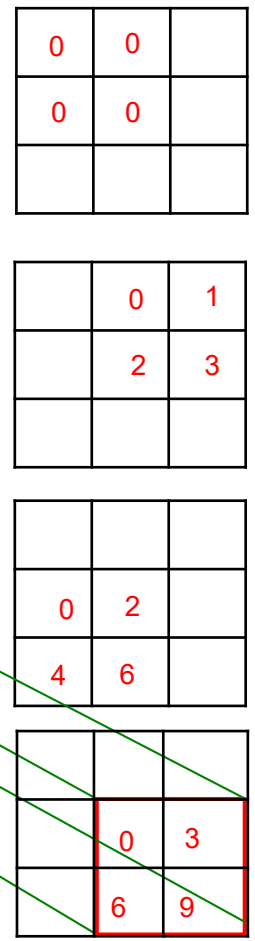
- Conv2Dtranpose: it is <u>opposite</u> of conv2D
  - Input =[2,2], kernel = [2,2], stride = [1,1] → output = [3,3]



input

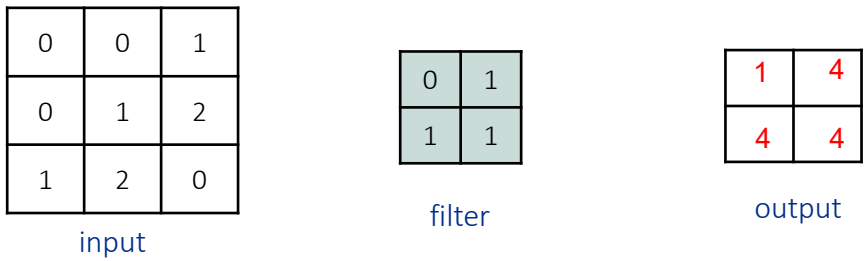
X

filter

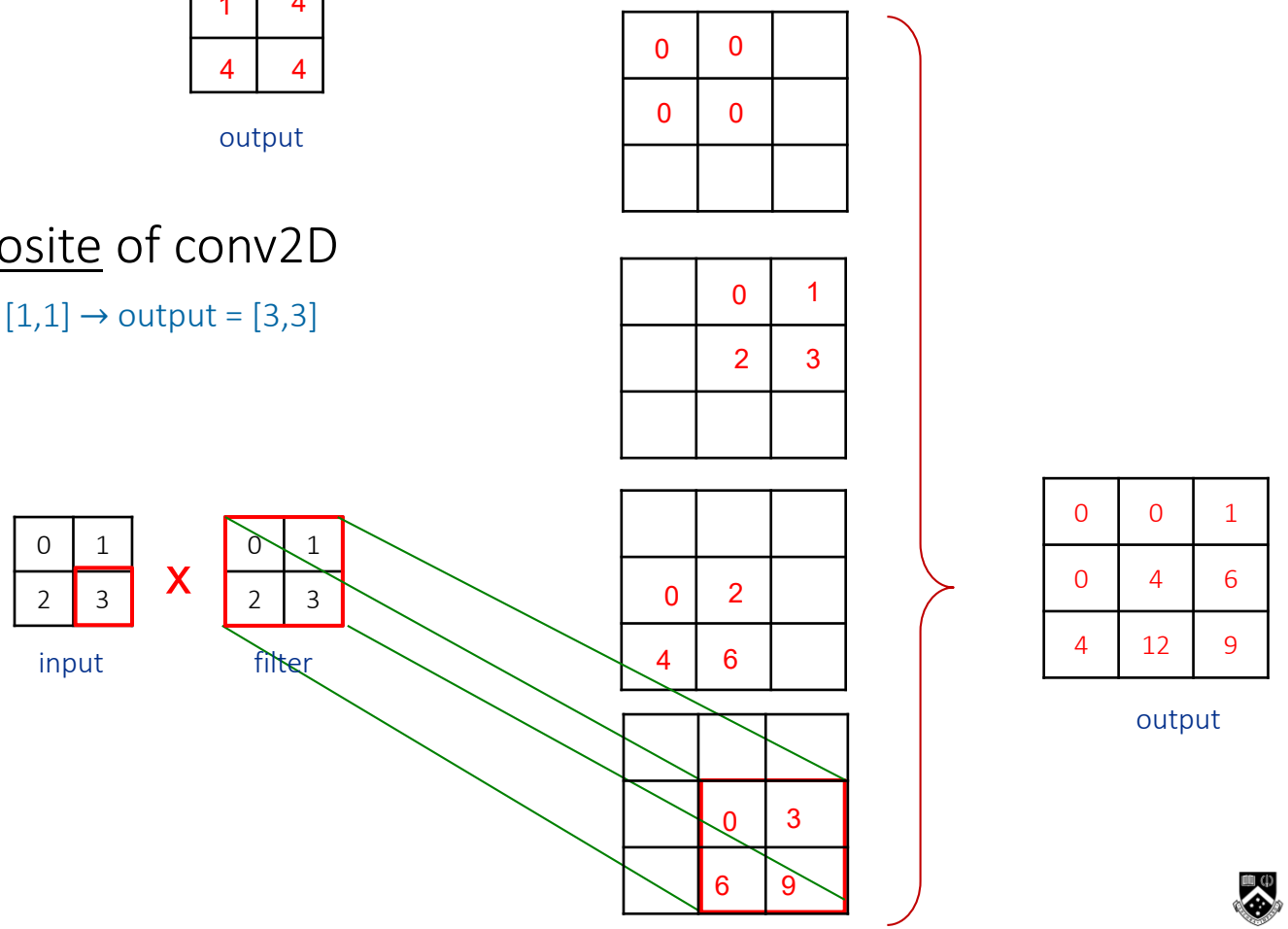


Introducing Conv2DTranspose

- Standard Conv2D
  - **Conv2D:** input = [3,3], kernel = [2,2], stride = [1,1] → output = [2,2]

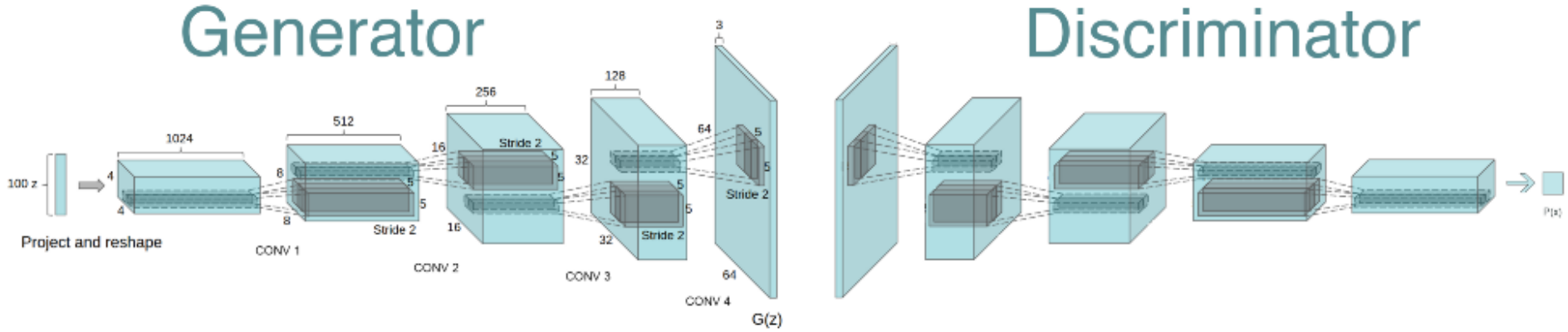


- Conv2Dtranpose: it is <u>opposite</u> of conv2D
  - Input =[2,2], kernel = [2,2], stride = [1,1] → output = [3,3]



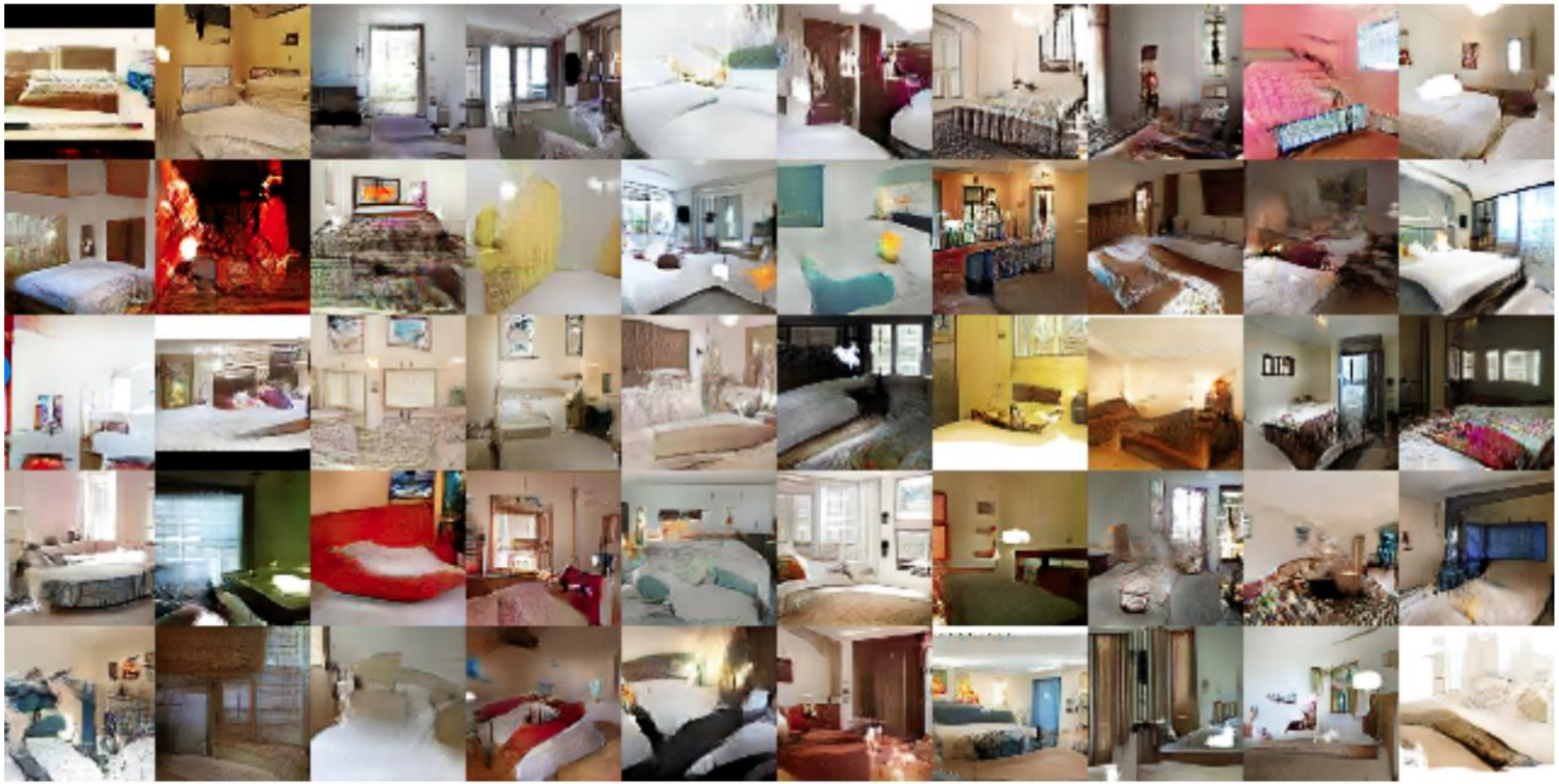
DCGAN architecture

Radford et al, 2015



- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batch normalization in both the generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output which uses tanh.
- Use LeakyReLU activation in the discriminator for all layers.

DCGAN's generated images



Generated bedroom after 5 epochs

Radford et al, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks", 2015

DCGAN: arithmetic on latent space

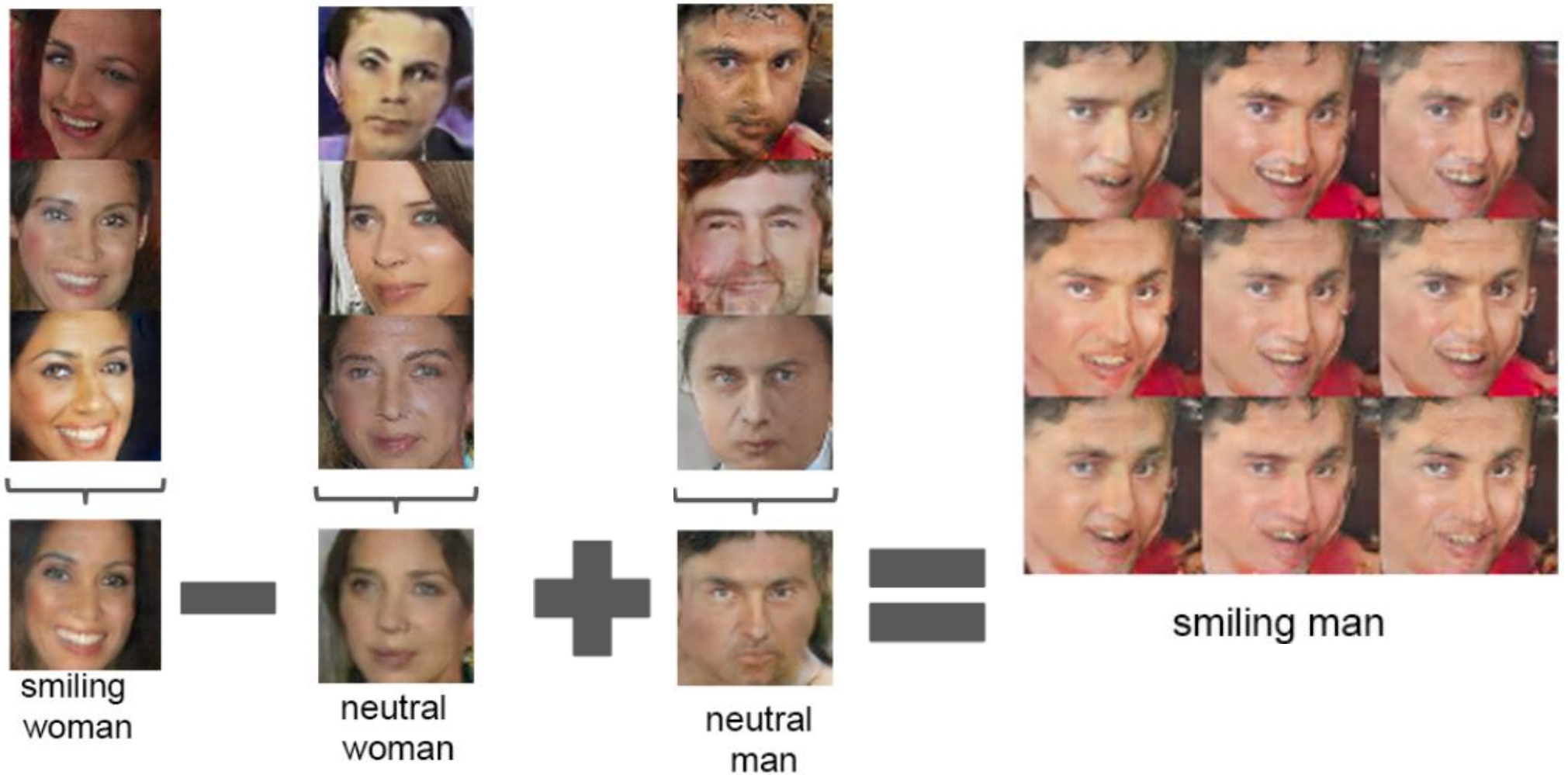


Figure 7: Vector arithmetic for visual concepts. For each column, the Z vectors of samples are averaged. Arithmetic was then performed on the mean vectors creating a new vector Y. The center sample on the right hand side is produce by feeding Y as input to the generator. To demonstrate the interpolation capabilities of the generator, uniform noise sampled with scale +-0.25 was added to Y to produce the 8 other samples. Applying arithmetic in the input space (bottom two examples) results in noisy overlap due to misalignment.

DCGAN: arithmetic on latent space

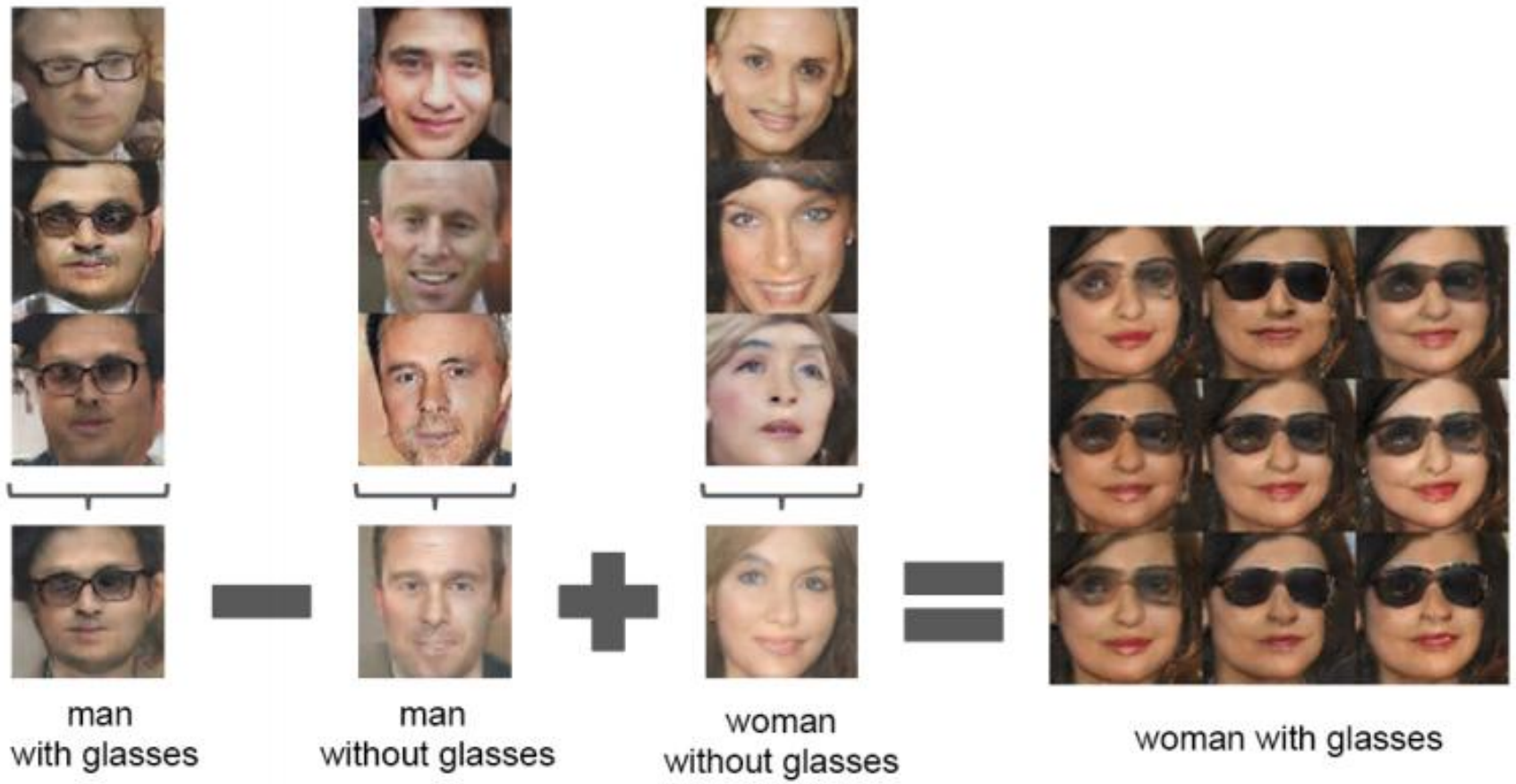
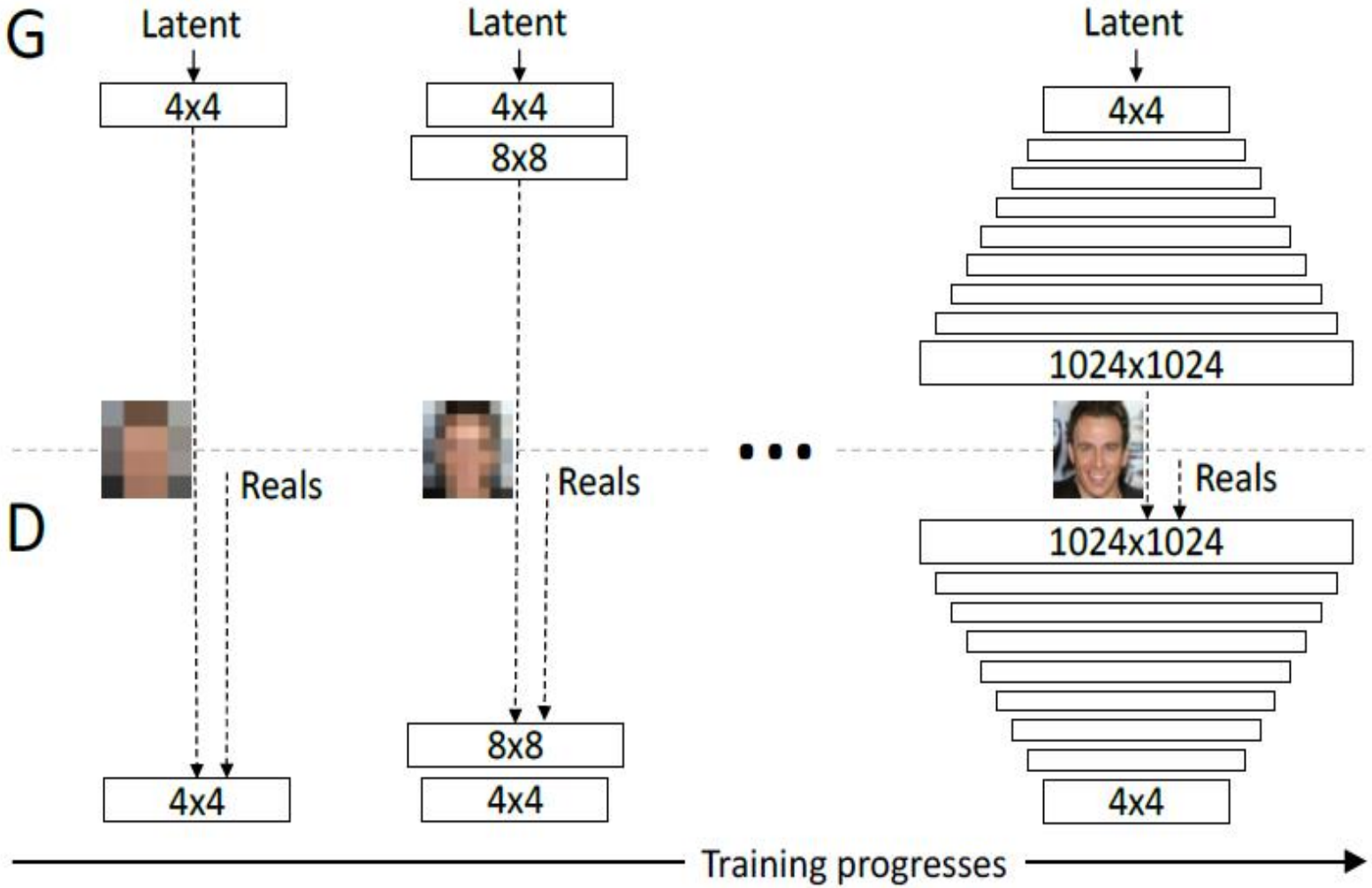


Figure 7: Vector arithmetic for visual concepts. For each column, the Z vectors of samples are averaged. Arithmetic was then performed on the mean vectors creating a new vector Y. The center sample on the right hand side is produce by feeding Y as input to the generator. To demonstrate the interpolation capabilities of the generator, uniform noise sampled with scale +-0.25 was added to Y to produce the 8 other samples. Applying arithmetic in the input space (bottom two examples) results in noisy overlap due to misalignment.

Progressive growing of GAN

(Kerras et. al NVIDIA, 2017)



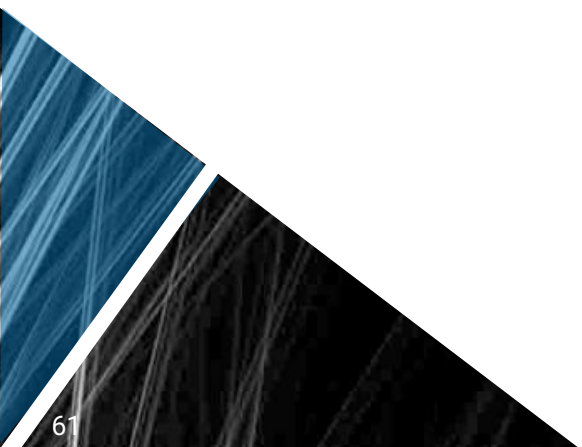


Further Reading Recommendation

- Hands-On: chapter 17
- Deep learning: chapter 20
- DC GAN: https://arxiv.org/pdf/1511.06434.pdf
- Wasserstein GAN: https://arxiv.org/pdf/1701.07875.pdf
- Wasserstein GAN with Gradient Penalty: https://arxiv.org/pdf/1704.00028.pdf
- Cycle GAN: https://arxiv.org/pdf/1703.10593.pdf
- Progressive GAN: https://arxiv.org/pdf/1710.10196.pdf



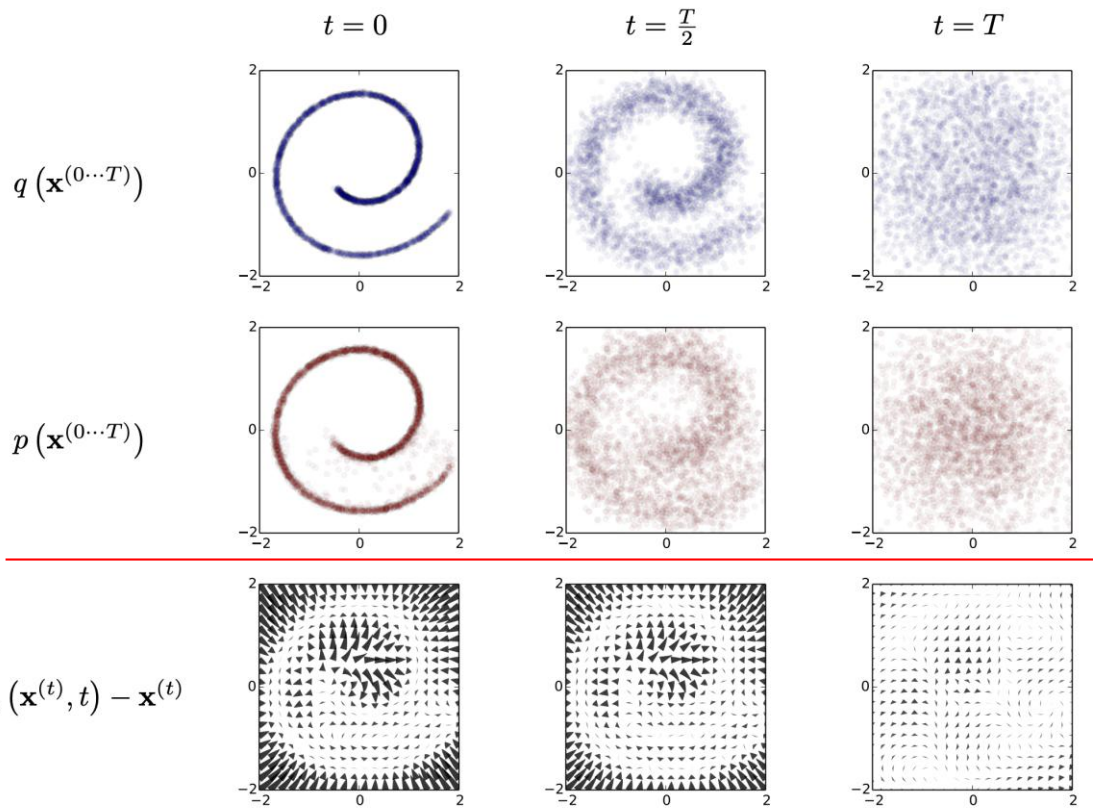
Diffusion Models



MONASH University

Motivation

- Non-equilibrium thermodynamics



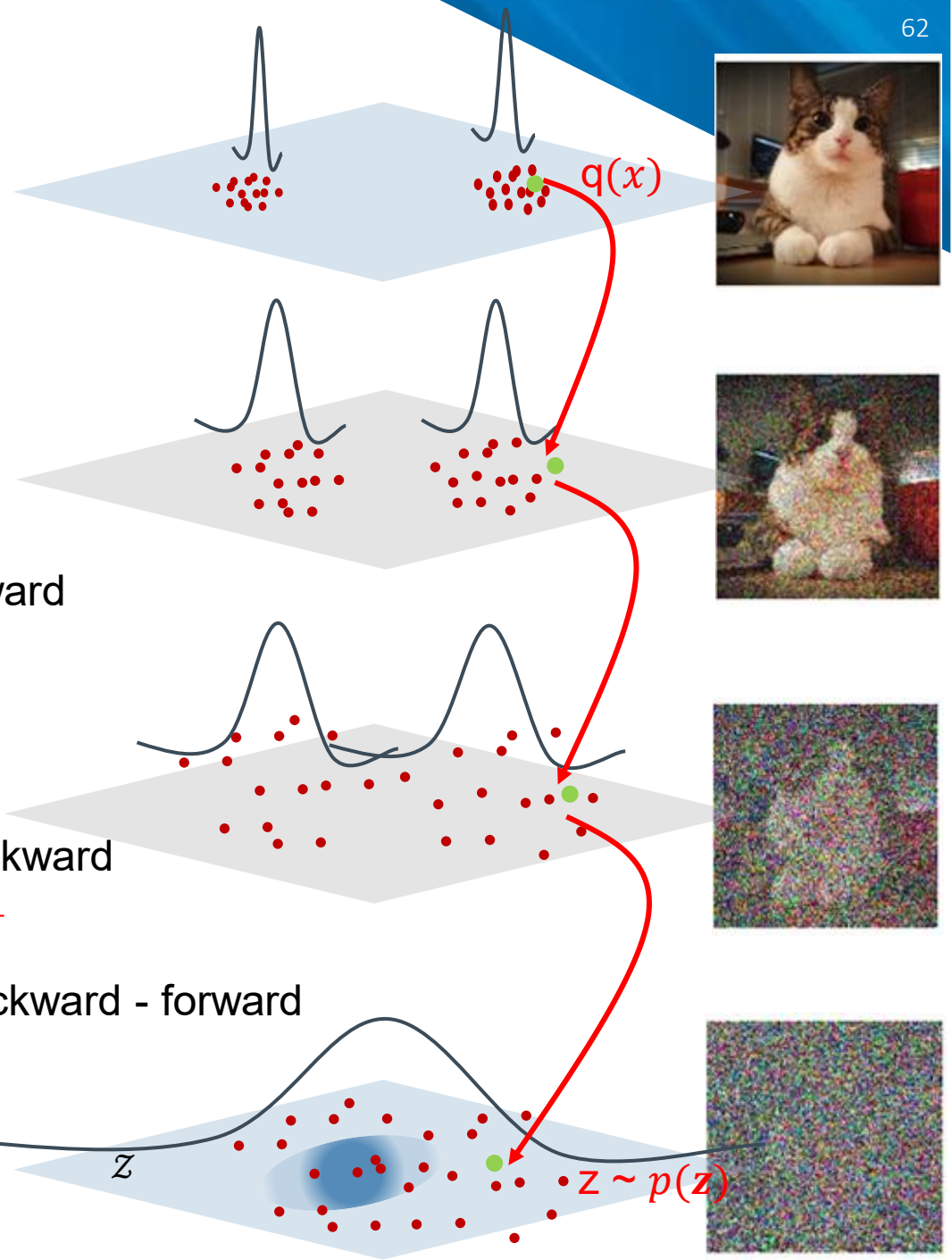
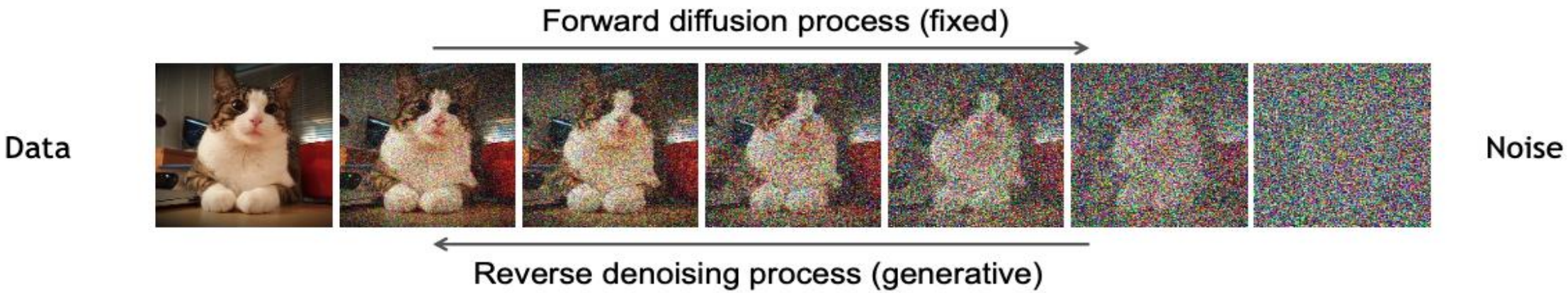


Image credit: Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015

General Diffusion Framework

- Forward diffusion process:
  - Adding noise to ‘destruct’ the image
- Backward (reverse) diffusion process
  - Denoising to reconstruct the image
  - Then, used as generator



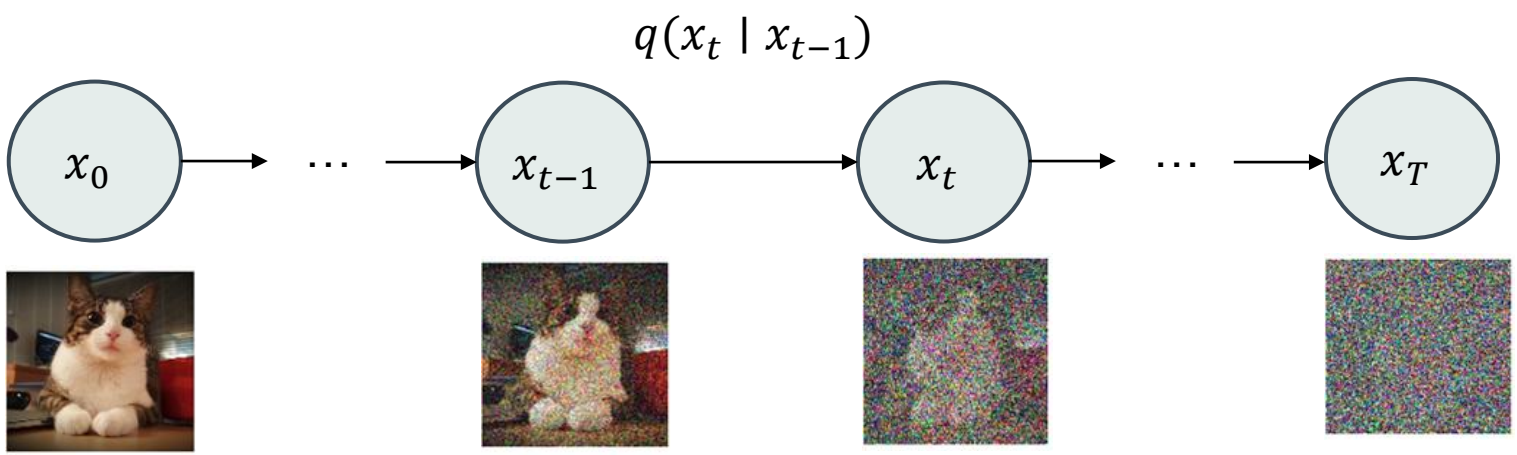
Key papers:
Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015
Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020
Song et al., Generating Modeling by Estimating Gradients of the Data Distribution, NeurIPS 2020

Image credit: https://cvpr2023-tutorial-diffusion-models.github.io/

Forward Diffusion Process

Formulated as Markov chain with T steps (usually large)

- At time 0:
  - Sample data point x_0 from real data distribution $q(x)$ (i.e., from training data)
- At step t:
  - $x_t = x_{t-1} + \text{Gaussian noise}$
  - x_t is now distributed according to $q(x_t|x_{t-1}) = \mathcal{N}(\cdot|\ \mu_t, \Sigma_t)$ with $\mu_t = \sqrt{1-\beta_t}x_{t-1}, \Sigma_t = \beta_t\ \boldsymbol{I}$ and $0 < \beta_t < 1$



Forward Diffusion Process

Image adapted from Ho et al. "Denoising Diffusion Probabilistic Models", NeurIPS, 2020.

Forward Diffusion Process

Formulated as Markov chain with T steps (usually large)

- At time 0:
  - Sample data point x_0 from real data distribution $q(x)$ (i.e., from training data)
- At step t:
  - $x_t = x_{t-1} + \text{Gaussian noise}$
  - x_t is now distributed according to $q(x_t|x_{t-1}) = \mathcal{N}(\cdot|\ \mu_t, \Sigma_t)$ with $\mu_t = \sqrt{1-\beta_t}x_{t-1}, \Sigma_t = \beta_t\ \boldsymbol{I}$ and $0 < \beta_t < 1$

- Question 1: How to sample x_t from $\mathcal{N}\left(x_t; \mu_t = \sqrt{1-\beta_t}x_{t-1}, \Sigma_t = \beta_t\ \boldsymbol{I}\right)$?
- Answer: use reparameterization trick
  - Sample randomness comes from the prior distribution $z = \mu + \sigma \cdot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$ is equivalent with $z \sim \mathcal{N}(\mu, \sigma)$

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_{t-1}$$

Image credit: Ho et al. “Denoising Diffusion Probabilistic Models”, NeurIPS, 2020.

Forward Diffusion Process

Formulated as Markov chain with T steps (usually large)

- At time 0:
  - Sample data point x_0 from real data distribution $q(x)$ (i.e., from training data)
- At step t:
  - $x_t = x_{t-1} + \text{Gaussian noise}$
  - x_t is now distributed according to $q(x_t|x_{t-1}) = \mathcal{N}(\cdot|\ \mu_t, \Sigma_t)$ with $\mu_t = \sqrt{1-\beta_t}x_{t-1}, \Sigma_t = \beta_t\ \boldsymbol{I}$ and $0 < \beta_t < 1$

- Question 2: Is there an analytical form for x_t directly from x_0?
- Answer: yes, $x_t \sim q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\boldsymbol{I}\right)$
  - define $\alpha_t = 1-\beta_t$, $\bar{\alpha}_t = \prod_{s=0}^{t}\alpha_s$ where $\epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(0, \boldsymbol{I})$ one can prove that:

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_{t-1}$$
$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2}$$
$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\bar{\epsilon}_0$$

where $\bar{\epsilon}_{t-2}$ is sampled a sum of two Gaussians, which is another Gaussian.

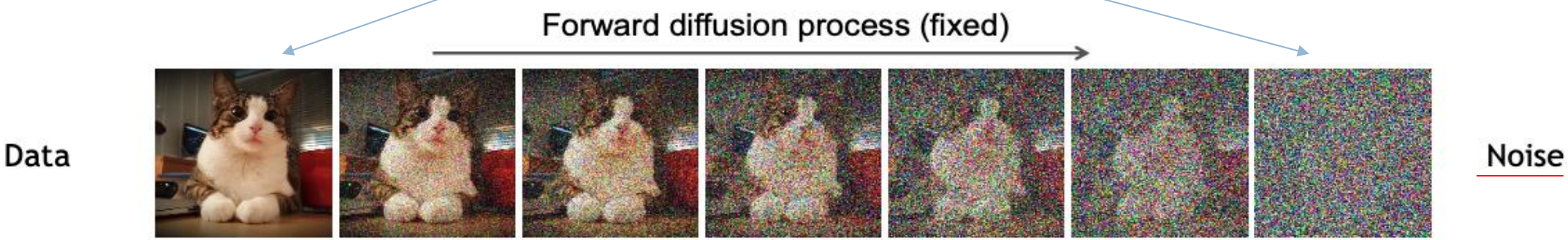
Image credit: Ho et al. “Denoising Diffusion Probabilistic Models”, NeurIPS, 2020.

Forward Diffusion Process

Importantly

$$x_t \sim q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\boldsymbol{I}\right)$$

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \text{ where } \epsilon \sim N(\mathbf{0}, \boldsymbol{I})$$



$\bar{\alpha}_t \to 0$, as $t \to \infty$, therefore

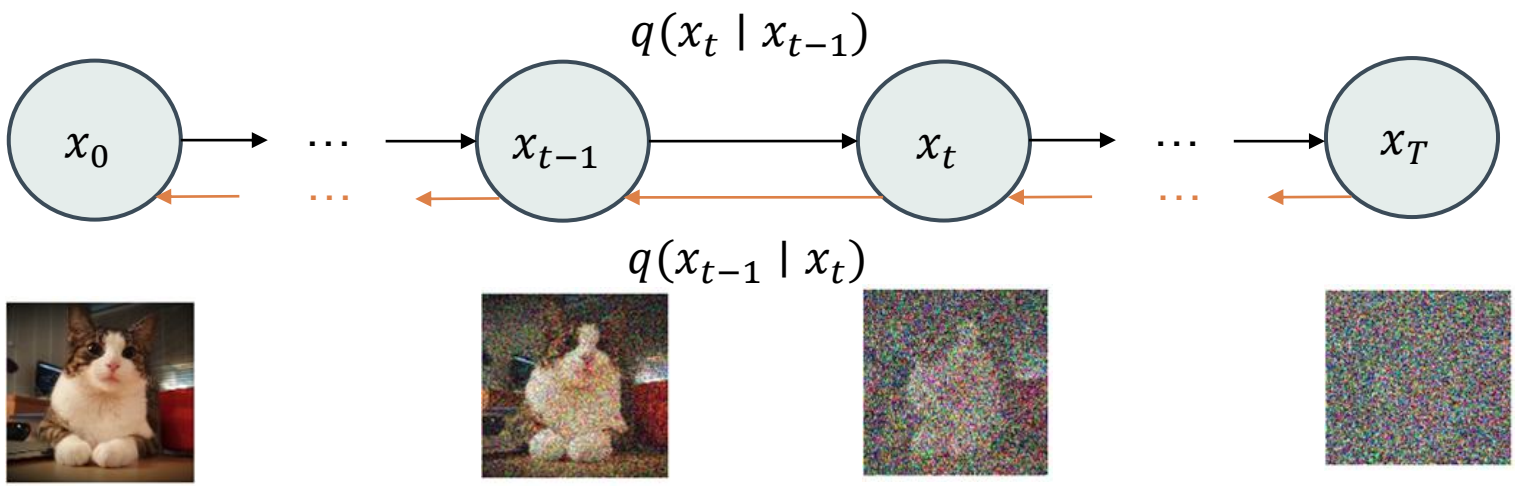
$$x_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \text{as } t \to \infty$$

Exercise: prove this!

Backward Diffusion Process

Iteratively denoise from time step T to 0

- At step T:
  - Sample a random vector x_T from $\mathcal{N}(0, \boldsymbol{I})$
- At step t:
  - $x_{t-1} = x_t - \text{amount of noise}$
  - x_{t-1} is now distributed according to $q(x_{t-1}|x_t)$



Backward Diffusion Process

Image credit: Ho et al. “Denoising Diffusion Probabilistic Models”, NeurIPS, 2020.

Backward Diffusion Process

Iteratively denoise from time step T to 0

- At step T:
  - Sample a random vector x_T from $\mathcal{N}(0, \boldsymbol{I})$
- At step t:
  - $x_{t-1} = x_t \; - \; \text{amount of noise}$
  - x_{t-1} is now distributed according to $q(x_{t-1}|x_t)$

- Issue: $q(x_{t-1}|x_t)$ is intractable
- Solution (Ho' et al.): conditioning on x_0 so that $q(\, x_{t-1} \mid x_t, x_0 \,)$ is tractable
- Question: How to known $q(x_{t-1}|x_t)$ (or $q(\, x_{t-1} \mid x_t, x_0 \,)$) so that we can sample x_{t-1}?
- Answer: Approximate by $p_\theta(x_{t-1}|x_t)$ which is a neural network parameterized by $\boldsymbol{\theta}$.
- Question: Can we get the $p_\theta(x_0)$?
- Answer: $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$ where $p_\theta(\, x_{t-1} \mid x_t \,) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t) \,)$

How to train diffusion models?

- Fix a forward process
  - Fix β_t, recall $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=0}^{t} \alpha_s$, and $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \sim q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)\boldsymbol{I}\right)$
  - Ideally, start from $x_T \sim N(0, I)$, we want $q(x_{t-1} \mid x_t)$ to generate new sample, but it is intractable!
  - Solution (Ho' et al.): it turns out that conditioning on x_0 then:
    - $q(x_{t-1}|x_t, x_0)$ is tractable, indeed is a Gaussian with mean $\tilde{\mu}_t(x_t, x_0)$ and covariance $\sigma_t^2 \boldsymbol{I}$
    - But x_0 is problematic for generation as we don't have it during generation process (why?)
    - Good news: maximise its expected negative log-likelihood, $E[-\ln(x_0)]$ yields a tractable variational distribution $p_\theta(x_{t-1} \mid x_t)$ to approximate $q(x_{t-1}|x_t, x_0)$ in the upper bound (similar to VAE) by minimising:

$$L_{t-1} = D_{KL}\left(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t)\right)$$

- How to parameterise and learn $p_\theta(x_{t-1}|x_t)$?
  - Assume it is a Gaussian with $\mu_\theta(x_t, t)$ and fixed covariance $\Sigma(x_t, t) = \sigma_t^2 \boldsymbol{I}$
    - Empirically, both $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \frac{1-\bar{\alpha}_t}{1-\bar{\alpha}_t}\beta_t$ works well
  - Since both distributions in the argument are Gaussians with same covariance $\sigma_t^2 \boldsymbol{I}$

$$L_{t-1} = D_{KL}\left(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t)\right) = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2\right]$$

  - Since $\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$ explicitly, introduce a 'denoise' network $\epsilon_\theta(x_t, t)$ so that $\mu_\theta(x_t, t) = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$, hence now: $L_{t-1} = \mathbb{E}_{x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0,I)}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$

Training and Sampling Algorithms

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

At each training iteration:

- Sample x_0 from the training set
- Sample time step t and noise ϵ
- Train the U-Net with the denoising matching loss at time step t

- Sample a starting noise x_T from the Normal distribution
- Iteratively remove noise:
  - Predict amount of noise (mean) to be removed $\epsilon_\theta(x_t, t)$
  - Remove noise from x_t by subtracting the predicted noise (subtracting the mean)
  - Add small amount of noise to simulate sampling operation. $\sigma_t = \sqrt{\frac{\beta_t(1-\bar{\alpha}_t)}{1-\alpha_t}}$ or $\sigma_t = \sqrt{\beta_t}$
- Return the last step x_0

Forward Diffusion Process – Noise Scheduler

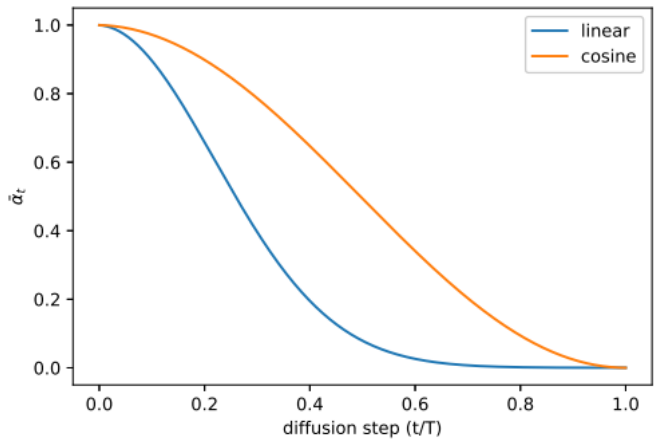
How to add noise in each step

Scheduling noise β_t is an art! Cannot add too much noise / too little noise at each step

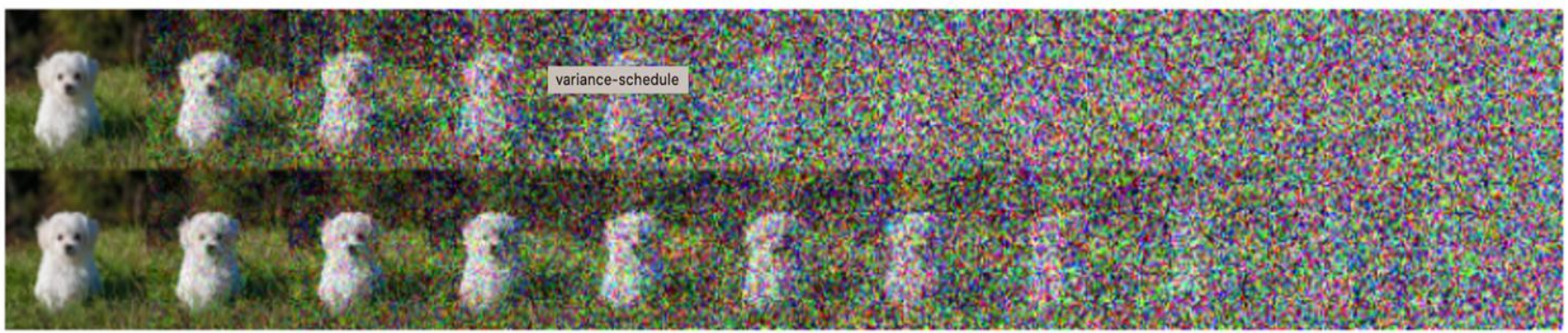
$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t \boldsymbol{I}\right)$$

Two common approaches

- Linear schedule: $\bar{\alpha}_t = \prod_{s=0}^{t}(1-\beta_s)$ where β_s is a constant
- Cosine schedule: $\bar{\alpha}_t = \frac{f(t)}{f(0)}$ where $f(t) = \cos\left(\frac{\frac{t}{T}+s}{1+s} \cdot \frac{\pi}{2}\right)^2$



$\bar{\alpha}_t$ throughout process



Latent samples from linear (top) and cosine (bottom) schedules. Samples in the second half of the linear schedule are almost noise, whereas those of cosine schedule are noising slowly.

Image source: Nichol, Alexander Quinn, and Prafulla Dhariwal. "Improved denoising diffusion probabilistic models.", ICML, 2021

Architecture and Time Embedding

The denoising network $\epsilon_\theta(x_t, t)$:

- Uses U-Net architectures with residual blocks and self-attention layers
- Input: previous sample x_t and time embedding
- Output: amount of noise to be removed $\epsilon_\theta(x_t, t)$

Time Embedding: Represent time step $\boldsymbol{t}$ by a vector via FC layers and embed to U-Net (concatenate to each intermediate representation of U-Net ϵ_θ with input x_t)

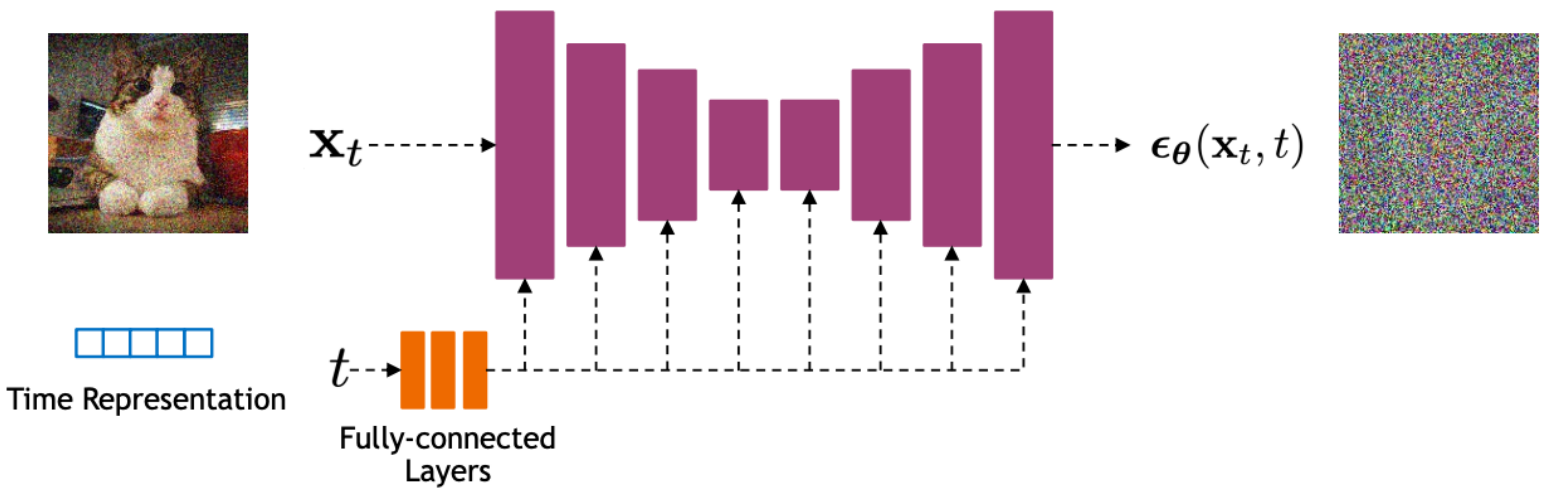


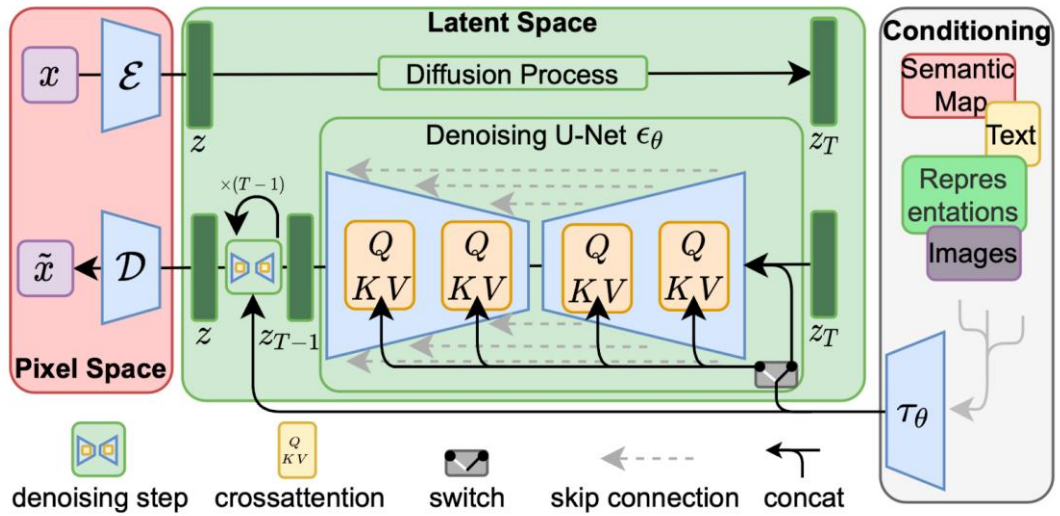
Image credit: https://cvpr2022-tutorial-diffusion-models.github.io/

Latent Diffusion Model

The architecture empowers Stable Diffusion, ImageGen, Midjourney

Three important contributions:

- Perceptual image compression via Encoder $\mathcal{E}$ and Decoder $\mathcal{D}$
- Latent diffusion process with denoising U-Net ϵ_θ
- Conditioning mechanism so that we can generate images by a prompt



Objective function:

- DM: $\mathbb{E}_{x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0,I)}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$ where $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$
- LDM: $\mathbb{E}_{z_0 \sim \mathcal{E}(x_0), \epsilon \sim \mathcal{N}(0,I)}[\|\epsilon - \epsilon_\theta(z_t, t)\|^2]$ where $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ and $z_0 \sim \mathcal{E}(x_0)$ is sampled from Encoder $\mathcal{E}$
- Conditional LDM: $\mathbb{E}_{z_0 \sim \mathcal{E}(x_0), \epsilon \sim \mathcal{N}(0,I)}[\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(y))\|^2]$ where y is condition (i.e., a caption/sentence) and τ_θ is a pre-trained domain encoder

Attention mechanism at layer $\boldsymbol{i}$:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

- $Q = W_Q^{(i)}\varphi_i(z_t), K = W_K^{(i)}\tau_\theta(y), \mathrm{V} = W_V^{(i)}\tau_\theta(y)$
- Here $\varphi_i(z_t)$ denotes a flattened intermediate representation at layer $\boldsymbol{i}$ of the U-Net with input z_t.

Image source: Rombach, Robin, et al. "High-resolution image synthesis with latent diffusion models.", CVPR, 2022

Diffusion Models

- A trendy and effective generative AI method
- Consists of two processes
  - Forward diffusion process: Iteratively adding noise to an input image, having closed-form and can be pre-computed
  - Backward diffusion process: Iteratively removing noise to reconstruct the input image, can be approximated by a U-Net model
- Learning is formulated as a probabilistic denoising process
  - Objective function is a denoising matching term
  - Need noise scheduler, time embedding, U-Net with self-attention to make it work
- Latent diffusion models works on the latent space
  - Diffusion is performed on the latent space hence deal with lower-dimensional space
  - Faster, and gives better overall results
  - Further enabling conditional generation – which is hugely applicable in real-world applications.

Further Recommended Readings

- Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020
- Song et al., Generating Modeling by Estimating Gradients of the Data Distribution, NeurIPS 2020
- Variational Autoencoders and Diffusion Models, by Dr. Ruiqi Gao from Google DeepMind
- What are Diffusion Models? By Dr. Lilian Weng

Summary

- Introduction to generative models
  - Explicit versus implicit density estimation
  - What is a generative model?
  - Why do we need to be generative?
- Generative Adversarial Networks
  - Adversarial training and minimax two-player game
  - Formulation and optimal solution
  - Issues and challenges with GAN
- Diffusion Models

Thanks for your attention!

Question time

